

Redes de Computadores

Ch1: Computer networks & internet

tabla de contenido

- 1. [Internet visto con tuercas y tornillos](#)
- 2.

Introducción

Internet vista con tuercas y tornillos



Millones de dispositivos computacionales conectados donde

- Host: final del sistema
- Corriendo aplicaciones en los bordes del internet



Switch de paquetes : paquetes (trozos de paquetes)

- Routers y switcher



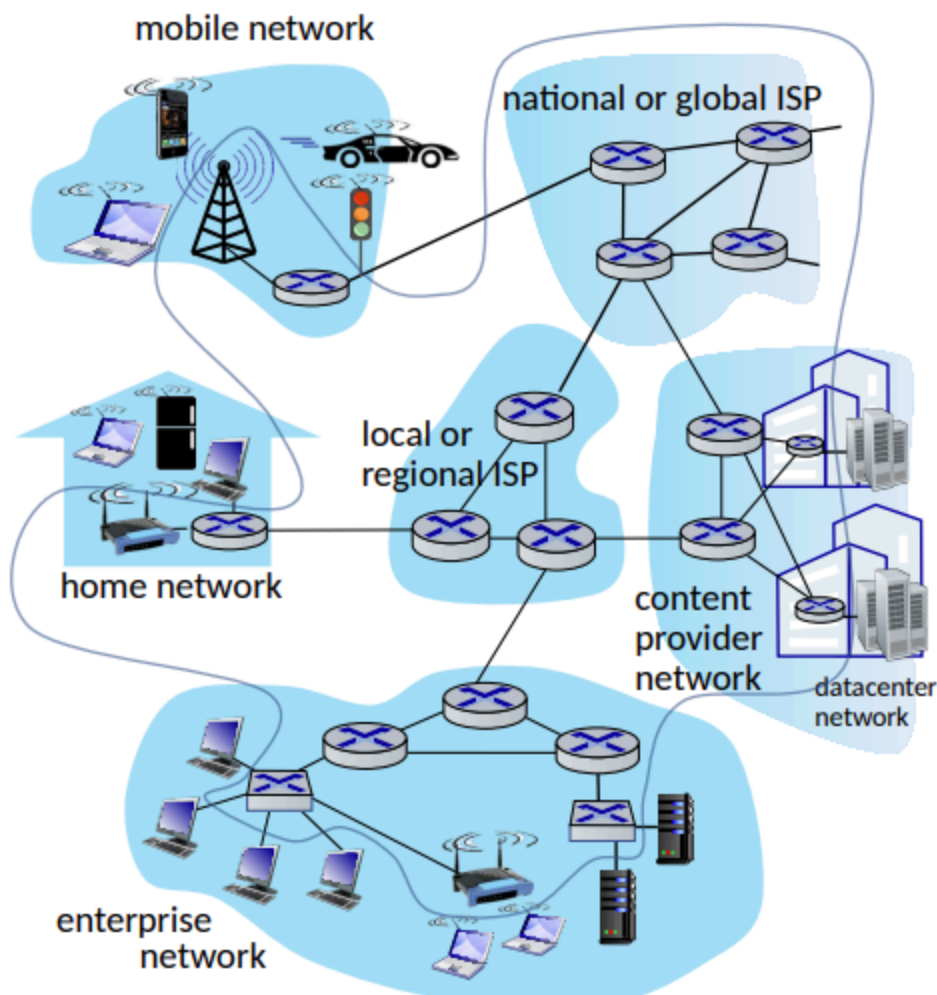
Canales de comunicación

- Fibra, cobre , radio, satélite, etc.
- Tasa de transmisión: ancho de banda



Redes

- Colección de dispositivos, routes y conexiones, administrada por una organización.



El Internet es la red de redes

- Son ISPs interconectados

Protocolos esta en todos lados

- Control de envío y recepción de mensajes
- HTTP, Streaming, Skype, etc..

Estándares del Internet

- RFC: solicitud de comentarios
- IETF: Ingenieros de fuerza de tarea del internet

Internet con vista a los servicios

Infraestructura que provee servicios para aplicaciones

- web, Streaming, Multimedia, Correo, etc..

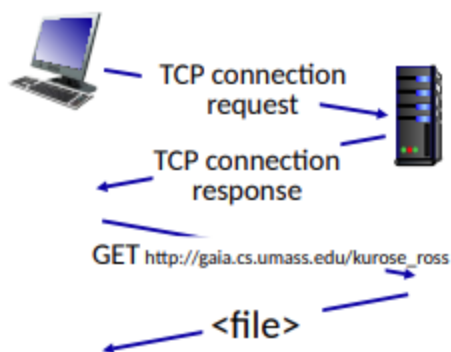
Provee de una interfaz de programación para aplicaciones distribuidas

- "Hooks", nos permite enviar y recibir aplicaciones para conectarnos y usar los medios de transporte del Internet
- Provee opciones de servicios, análogos a servicio postal

¿ Que es un protocolo?

Usado por computadores donde los protocolos manejan todo el Internet

Los protocolos definen el formato, orden de los mensajes enviados y recibidos entre entidades de redes, y acciones tomadas en transmisión de mensajes, recibirlo.



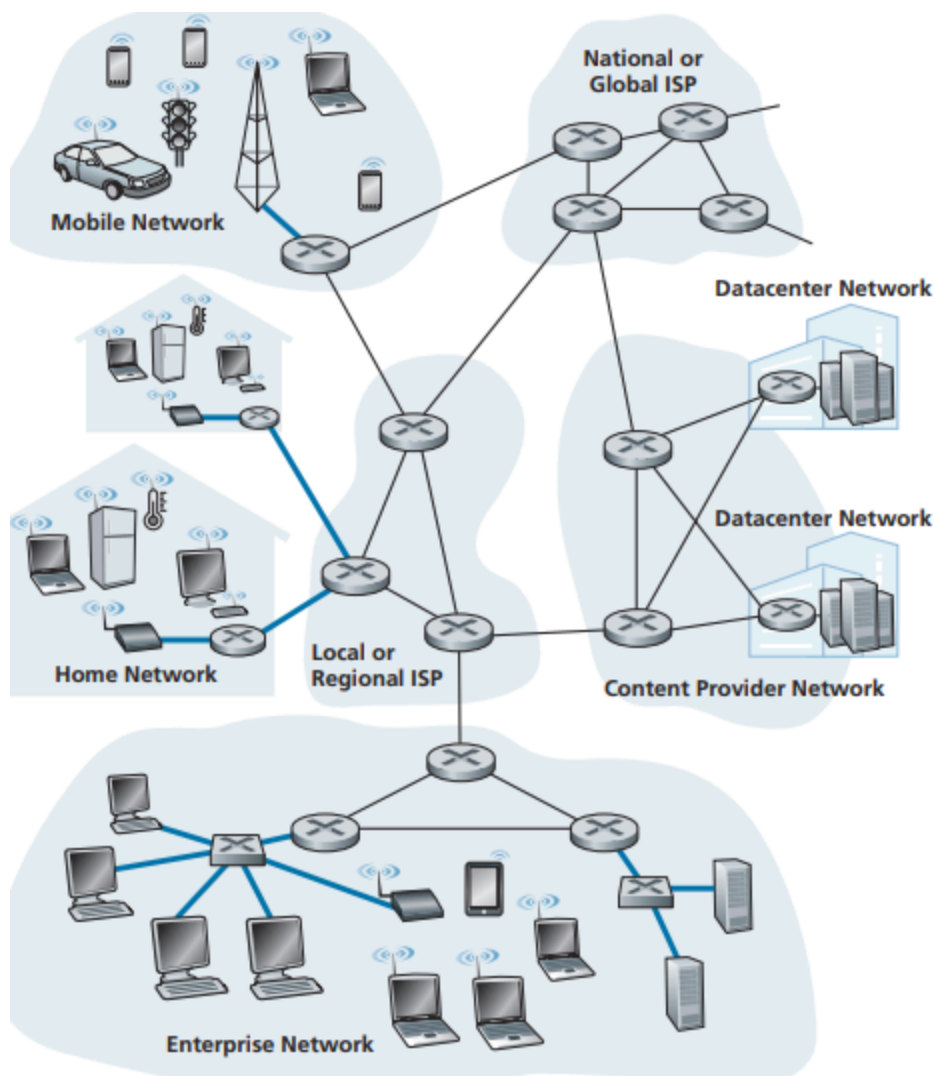
Network edge

Podemos referirnos a ellos como **host** y estos se dividirían en dos categorías **Cliente** y **Servidores**

- Cliente: suelen ser maquinas comunes, computadores, celulares, etc.
- Servidores: Suelen ser maquinas potentes o data centers.

Acceso a la red

Para conectarnos a la red y usamos usualmente un router el cual se conecta con los ISPs locales, estos "routers de bordes" nos permiten conectarnos con el exterior y saltar entre islas de redes



Acceso desde casa

Hay dos maneras de conectarse al Internet desde casa que predominan, que son DSL(Linea digital de suscripción) y cable.

DSL

Los DSL usan los cables de teléfonos preexistentes, estos se conectan mediante un modem, y llegan a la compañía de telecomunicaciones y mediante un multiplexor(DSLAM) que transforma los tonos de alta frecuencia en formato digital, entonces la compañía actúa también

como un ISP

- A high-speed downstream channel, in the 50 kHz to 1 MHz band
- A medium-speed upstream channel, in the 4 kHz to 50 kHz band
- An ordinary two-way telephone channel, in the 0 to 4 kHz band

This approach makes the single DSL link appear as if there were three separate links, so that a telephone call and an Internet connection can share the DSL link at

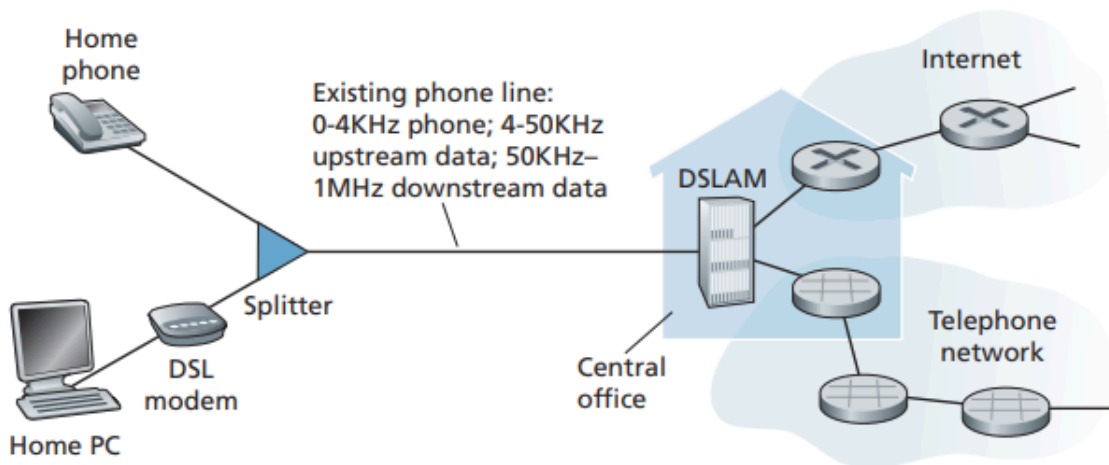


Figure 1.5 ♦ DSL Internet access

Este sistema tiene una desventajas:

- Conexión asimétrica (diferencia entre subida y bajada)
- Peor rendimiento mientras mas lejos este físicamente el módem de la oficina central

Cable

Este sistema usa los cables de televisión preexistentes, cada cable soporta entre 500 y 5000 casas conectadas, usando un sistema de cable coaxial para alcanzar cada hogar esto a veces se llama HFC (fibra híbrida coaxial) .

En la casa para conectarse se usa un módem de cable que va conectado por el puerto ethernet del computador el cual lleva finalmente a un CMTS similar al DSLAM en función, este tipo de conexión tiene una característica, los datos se envían por un cable compartido por lo cual la red se puede saturar, también se necesita coordinar la transmisión para evitar colisiones.

- Es asimétrico
- se puede sobrecargar el cable coaxial

- Puede haber colisión debido a que múltiples casas usan la misma red para conectarse

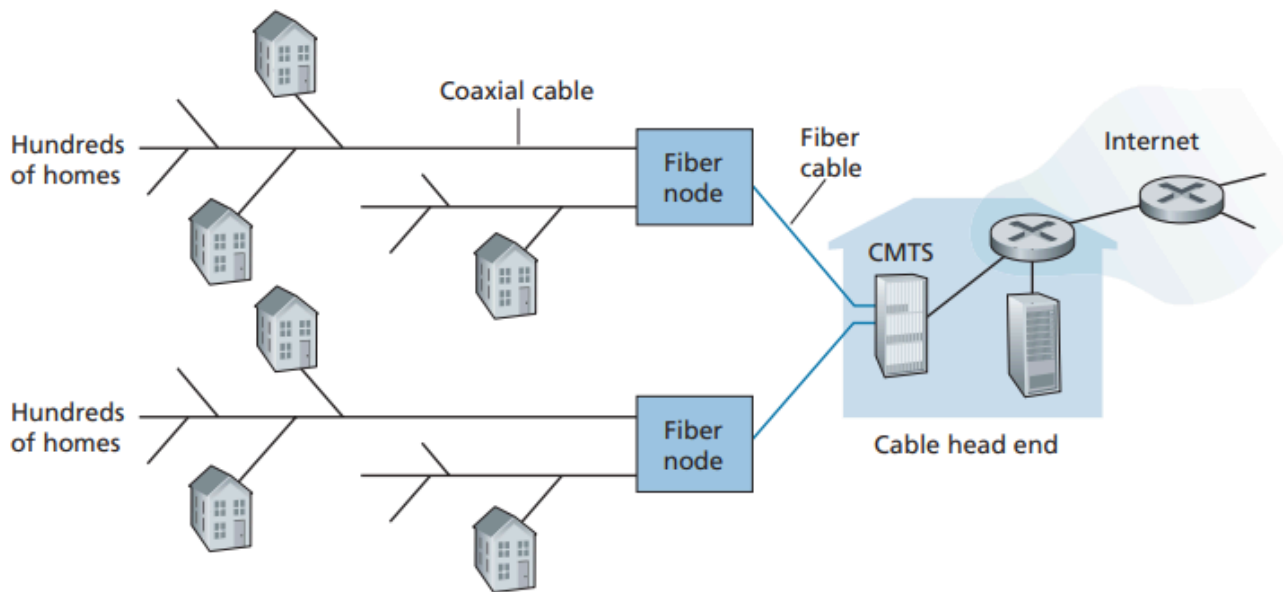
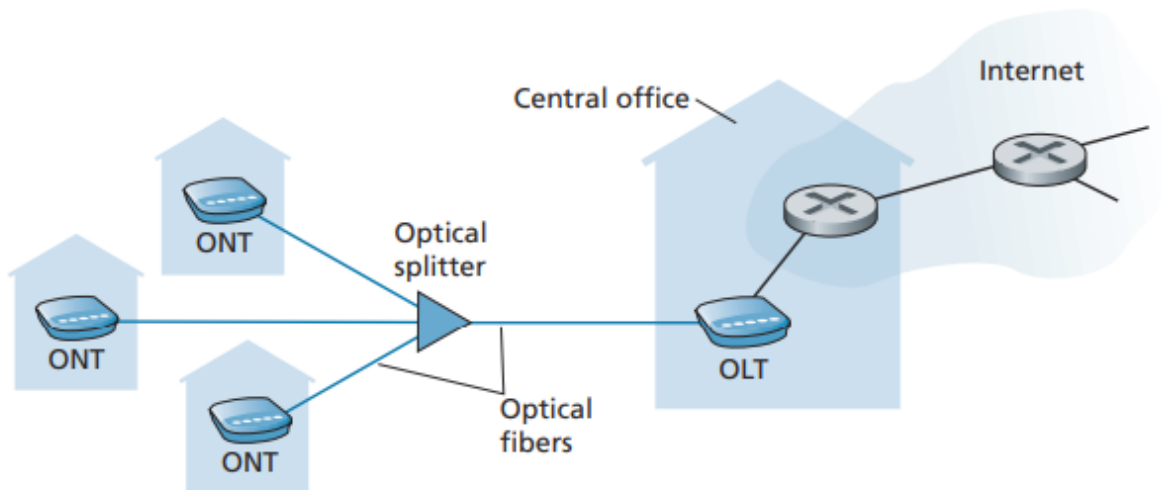


Figure 1.6 ♦ A hybrid fiber-coaxial access network

Fibra a casa (FTTH)

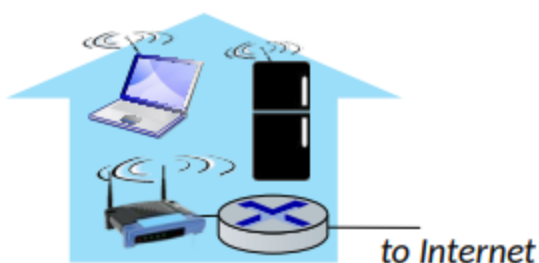
El concepto es tener un cable de fibra óptica que sale de la oficina central y va hacia las casas. En el sistema PON, cada casa tiene un terminador óptico de red que se conecta a un splitter del vecindario, habitualmente menos de 100 casas, luego va a un OLT que realiza la transformación de señales.



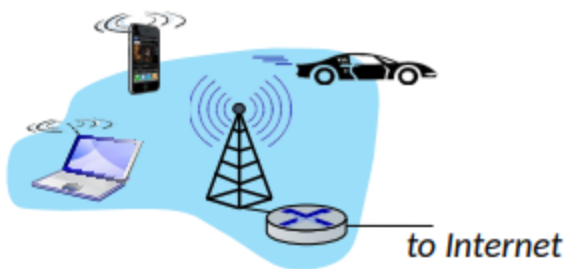
Acceso a la network mediante wireless

conectamos el end system al router al cual llamaremos punto de acceso, los separamos en dos

1. **Wireless local area networks(WLANs):** Tipicamente alrededor de edificios, 802.11b/g/(WiFi): 11, 54, 450 mbps de transmisión

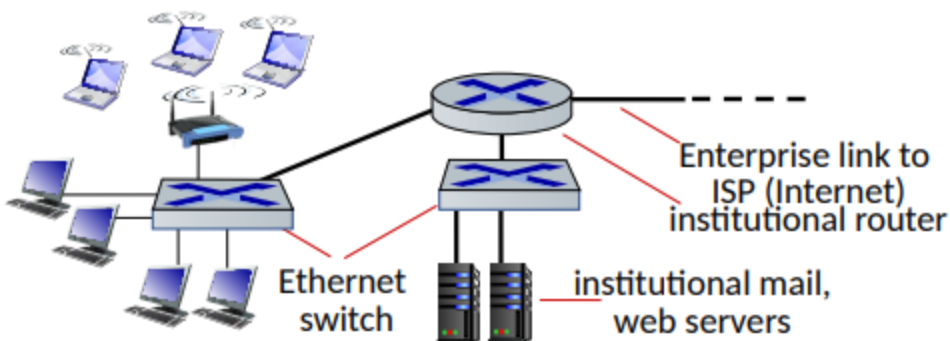


2. **Wide-area cellular access networks:** Lo provee los operadores móviles , 10Mbps, 4G y 5G



Red de empresas

- Compañías, universidades, etc.
- Mix de cables, wireless, switches y routers
- Ethernet y wifi



Data center red

Ancho de banda alto (10s to 100s gbps) conectando cientos o miles de servidores a si mismos y al Internet

Host: envíos de paquetes de data

host tiene la función de enviar:

- toman mensajes de la aplicación
- rompe en pequeños en pequeños chunks, conoce como paquetes de un largo de L bits
- transmite paquetes con un ratio de transmisión R
 - transmisión de conexión, capacidad o ancho de banda

$$PacketTransmissionDelay = tiempoNecesarioParaTransmitirLBitsEnElEnlace =$$

Enlace físicos

- bit: propagado entre transmisor y receptor par
- Enlace físico: lo que hay entre el transmisor y receptor
- Medio guiado: señal propagada en un medio solido por ejemplo un cable
- Medio no guiado: señal propagada libremente, ej Radio
- Twisted pair(TP): dos cables de cobre entrelazados
 - categoría 5: 100 Mbps, 1 Gbps ethernet
 - categoría 6: 10 Gbps ethernet
- Coaxial cable: dos conductores de cables concentricos, bidireccional, tiene múltiple frecuencia de los canales del cable y con unos 100 Mbps por canal
- Cables de fibra óptica: cable de cristal que llevan pulsos de luz, cada pulso es un bit, alta velocidad, bajo error ya que hay repetidores y es inmune a electromagnetismo
- Ondas de radio inalambrico: varias bandas del espectro electromagnético, no hay un cable físico, emisor "half-duplex", es afectado por el entorno
 - Wireless LAN(WiFi)

- Wide-area(4G)
- Bluetooth
- Ondas de microondas
- satélite

Network core

- malla de routers intercomunicados
- packet-switching: el servidor rompe los mensajes de las aplicaciones en paquetes
 - la red envía paquetes de un router al otro a través de enlaces hasta su destino

forwarding

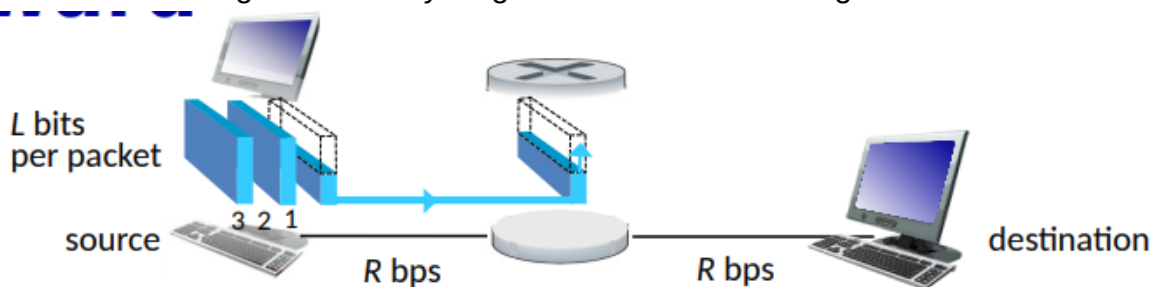
- aka switching
- acción local: mover los paquetes entrantes al enlace correcto para seguir su camino

Routing

- Acción global: determinar la ruta que tomara el paquete para llegar a su destino.
- algoritmo de ruta.

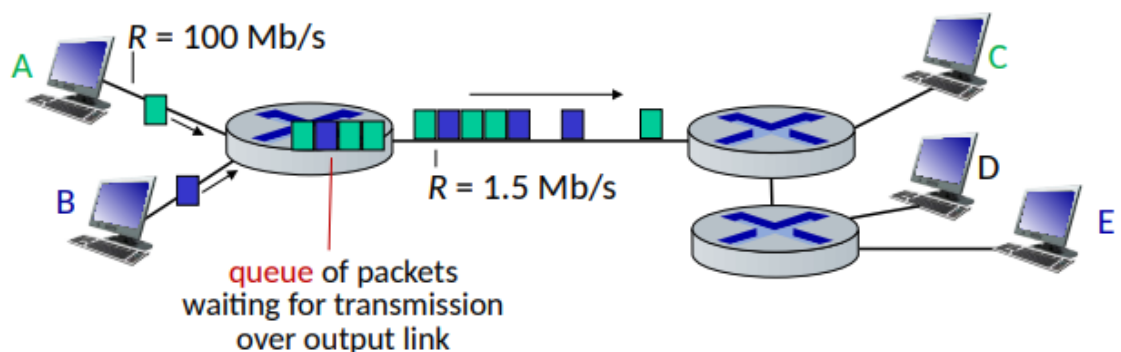
store and forward

- tiene delay
- paquetes enteros llegan al router y luego son transmitidos al siguiente enlace.



cola

La cola llega cuando el trabajo llega mas rápido de lo que puede ser trasladado.



Los paquetes se almacenaran en la cola pero si se sobrepasa la capacidad de buffer este generara perdida de paquetes

circuit switching

Es otra forma de transmitir datos, en donde se reserva un camino entre el origen y el destino para transmitir todo los datos, garantiza el rendimiento

Circuit switching: FDM and TPM

Frequency Division Multiplexing

Dividimos la frecuencia depende la cantidad de usuarios, si tenemos 4 usuarios, a cada uno les corresponde el 25% del ancho de banda

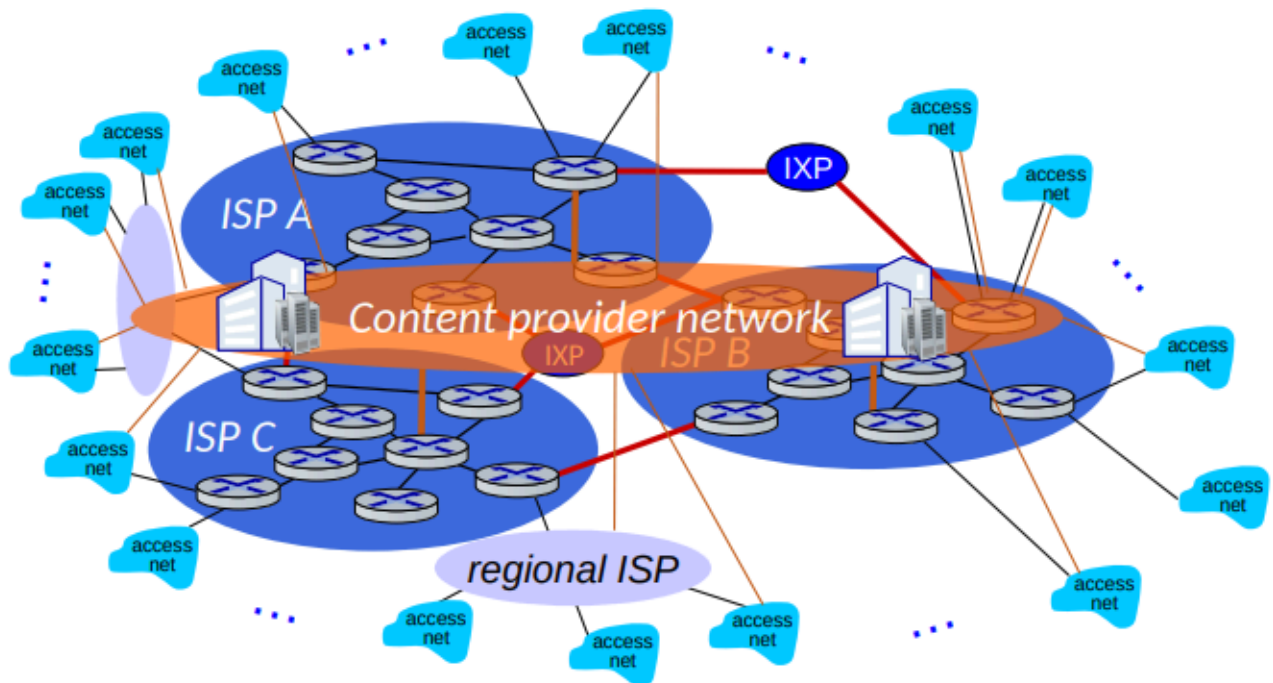
Time division Multiplexing

Usamos el tiempo para dividir a los usuarios, los usuarios cuenta con el 100% de la red por un tiempo limitado, si fueran 10 segundos y 4 usuarios, serian 2,5 segundos por usuario

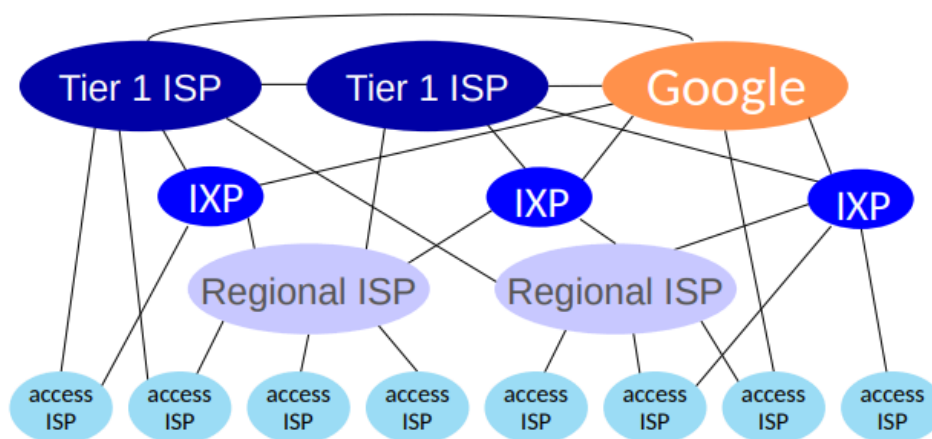
Packet switching vs circuit switching

- circuit switching: 10 usuarios activos
- Packet switching: podemos tener 35 usuarios activos, donde tenemos una probabilidad de 10 activos, este es mas eficiente.
 - Se consume en base a si el usuario esta activo, consume solo cuando esta en uso
 - hay una posible congestión, lo que provocaría perdida de datos y retraso en los paquetes

Estructura de la red de redes



tenemos que dividir la red para no colapsarla, usar proveedores locales que se conectan a proveedores mas grandes



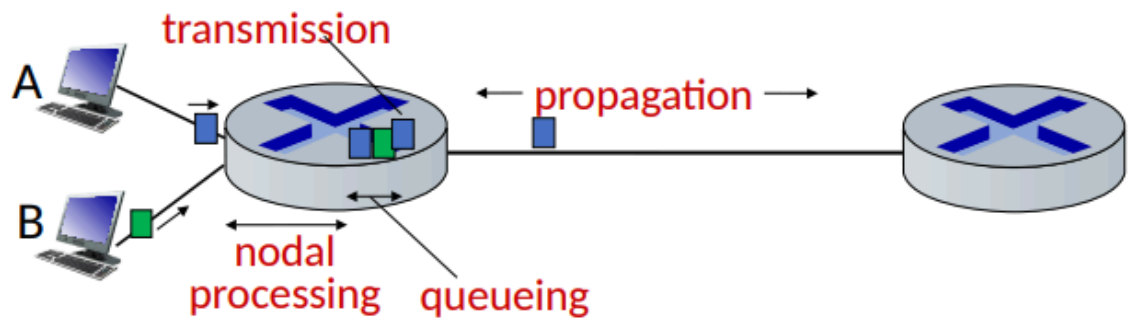
At “center”: small # of well-connected large networks

- “tier-1” commercial ISPs (e.g., Level 3, Sprint, AT&T, NTT), national & international coverage
- content provider networks (e.g., Google, Facebook): private network that connects its data centers to Internet, often bypassing tier-1, regional ISPs

Performance

Perdida de paquetes

los routers tienen buffers, cuando este buffer se llena esto genera perdida de los paquetes



$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

d_{proc} : nodal processing

- check bit errors
- determine output link
- typically < microsecs

d_{queue} : queueing delay

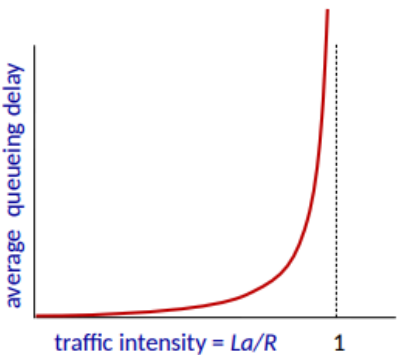
- time waiting at output link for transmission
- depends on congestion level of router

esta es la formula para calcular el delay del nodo, el delay de transmisión es distinto al de propagación

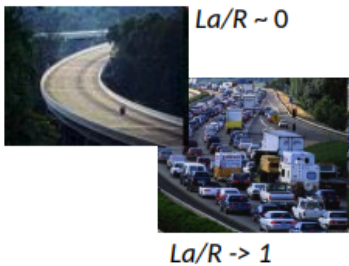
Delay de paquetes en cola

- a : average packet arrival rate
- L : packet length (bits)
- R : link bandwidth (bit transmission rate)

$\frac{L \cdot a}{R}$: $\frac{\text{arrival rate of bits}}{\text{service rate of bits}}$ "traffic intensity"



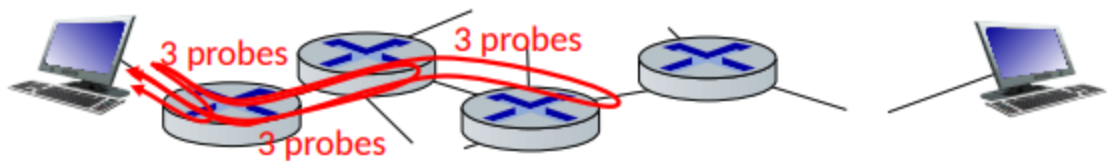
- $La/R \sim 0$: avg. queueing delay small
- $La/R \rightarrow 1$: avg. queueing delay large
- $La/R > 1$: more "work" arriving is more than can be serviced - average delay infinite!



es una manera de calcular el delay de la cola, si es cercano a 0, el delay es pequeño, si se acerca a 1 el delay es alto y si es mayor a 1 es infinito

Comando traceroute

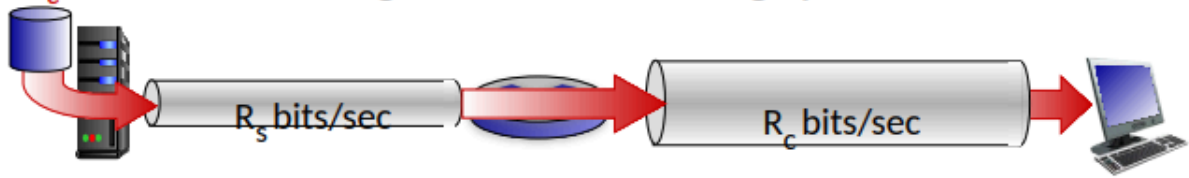
nos permite ver el delay, envía 3 paquetes al destino



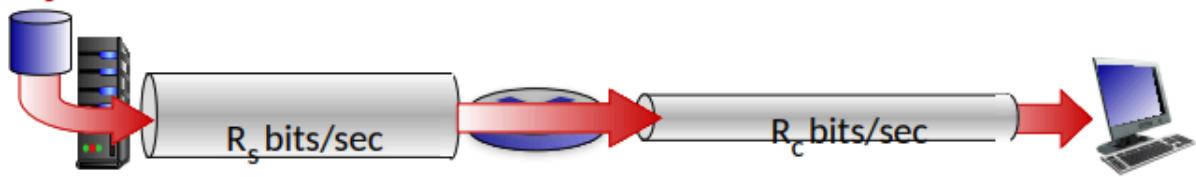
Throughput

es el ratio entre el que envía y el que recibe

$R_s < R_c$ What is average end-end throughput?



$R_s > R_c$ What is average end-end throughput?



bottleneck link

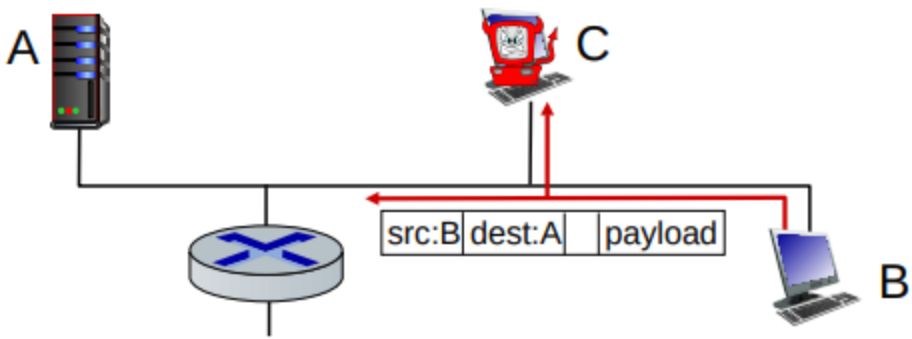
link on end-end path that constrains end-end throughput

aquí se genera un cuello de botella debido a que el mas pequeño pone el limite

Seguridad

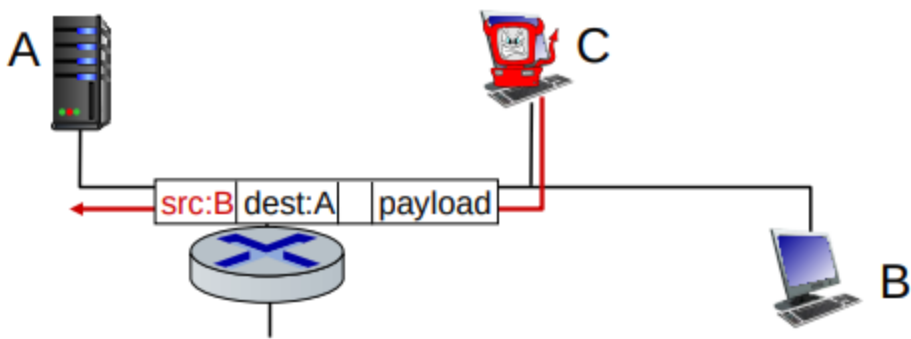
Sniffing de paquetes

la tarjeta en modo promiscuo puede leer todos los paquetes, escucha todo



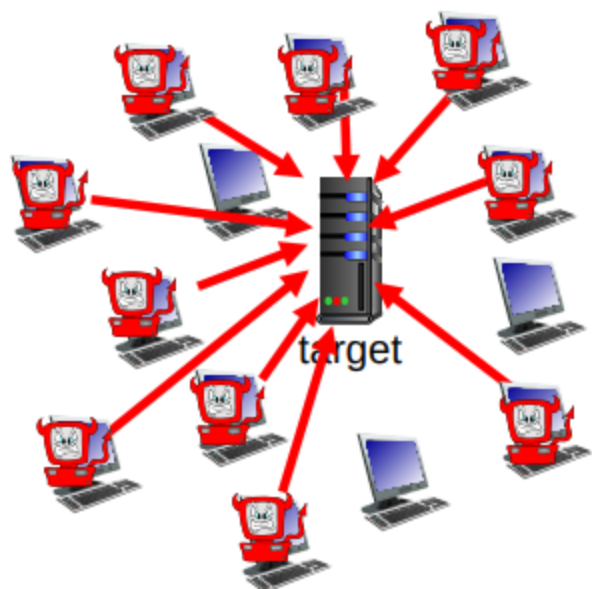
Falsa identidad

IP spoofing: inyecta un paquete con un origen de direccion falso



Denegación de servicios

sobrecargan el objetivo, mediante peticiones mal formadas



lineas de defensas

- autenticacion
- confidencialidad
- chequeo de integridad
- restriccion de acceso
- firewall

Capas de protocolos y servicios

Cada capa presenta un servicio

ticket (purchase)	<i>ticketing service</i>	ticket (complain)
baggage (check)	<i>baggage service</i>	baggage (claim)
gates (load)	<i>gate service</i>	gates (unload)
runway takeoff	<i>runway service</i>	runway landing
airplane routing	<i>routing service</i>	airplane routing

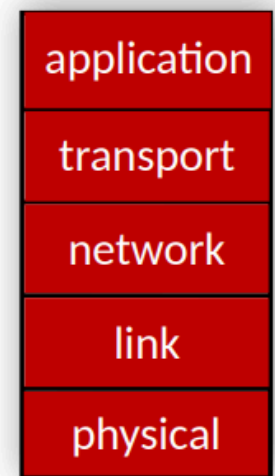
layers: each layer implements a service

- via its own internal-layer actions
- relying on services provided by layer below

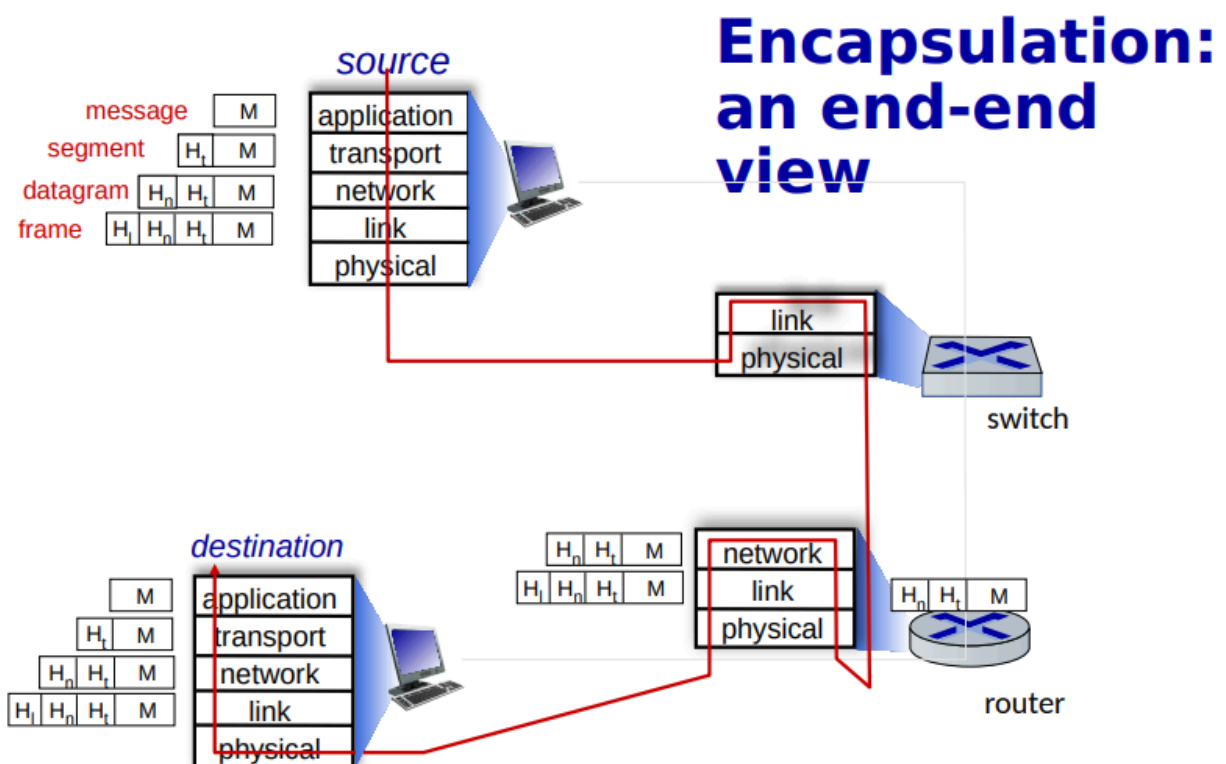
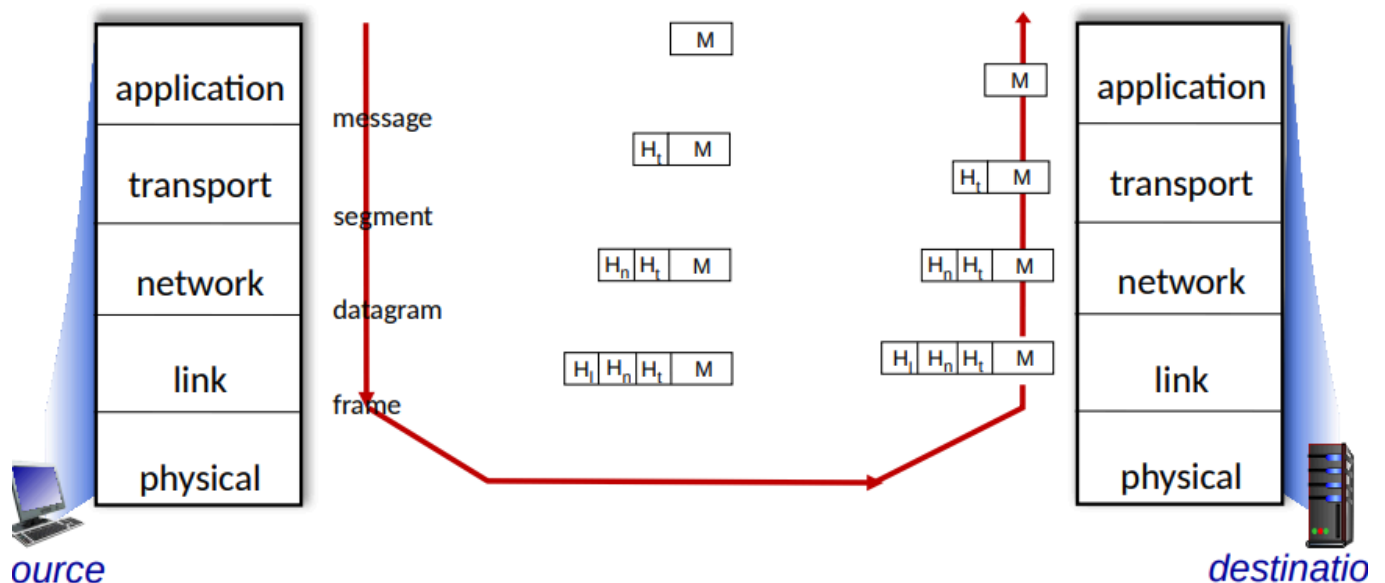
Usar el modelo de capas mejora la mantenci3n.

stack de las capas de los protocolos

- **application:** supporting network applications
 - HTTP, IMAP, SMTP, DNS
- **transport:** process-process data transfer
 - TCP, UDP
- **network:** routing of datagrams from source to destination
 - IP, routing protocols
- **link:** data transfer between neighboring network elements
 - Ethernet, 802.11 (WiFi), PPP
- **physical:** bits "on the wire"



Services, Layering and Encapsulation



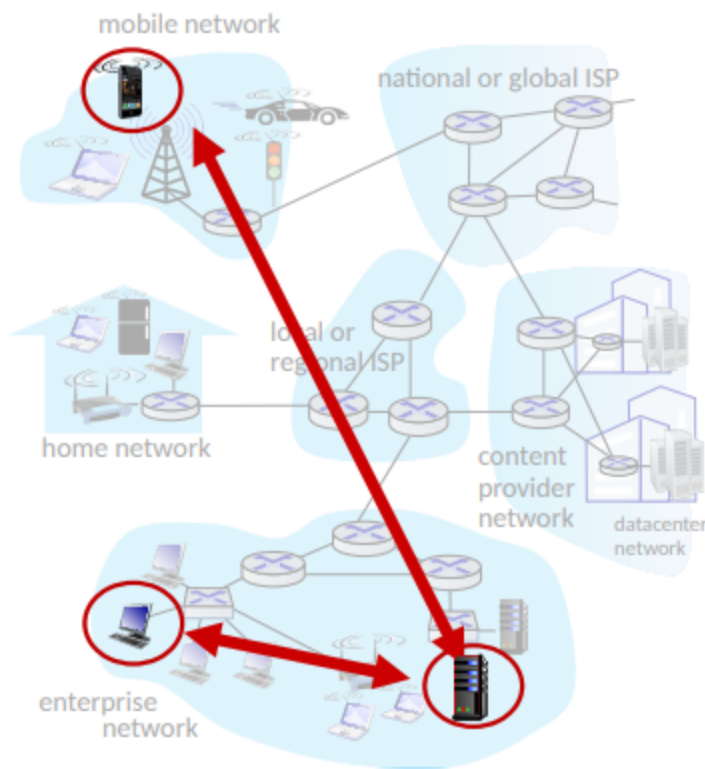
En resumen los protocolos tienen una estructura al enviarse "se arma de una manera", y al llegar al destino se "desarma" para extraer la información

ch2: capa de aplicación

Principios

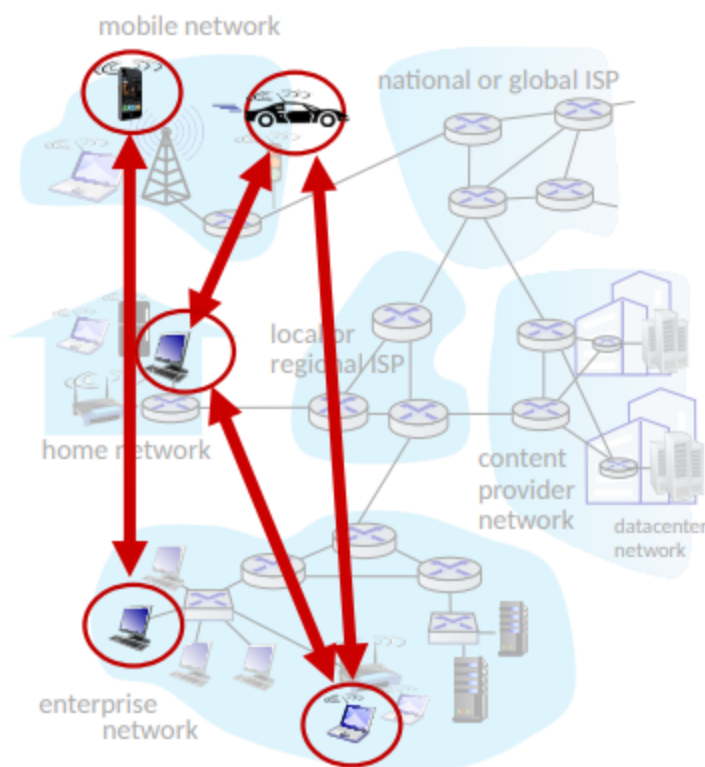
paradigma cliente-servidor

- **Host:** Ip estática, siempre hosteando y a veces en datacenters.
- **Cliente:** Ip dinámica, se contacta con el servidor, no se comunica con otros clientes.



P2P Arquitectura

- No siempre en servidor
- Sistema arbitrario de fin del sistema
- los pares hacen peticiones de un servicio al otro y otro por lo retorna
- cambia de ip



clients, servers

client process: process that
initiates communication
server process: process
that waits to be contacted

Socket

El socket es un análogo a una puerta, es una comunicación donde se reciben y se envían mensajes, es como una comunicación entre dos extremos donde no se interactúa con el exterior, tienen que estar en el mismo socket para comunicarse.

Proceso de direcciones

Para enviar algo necesitamos la ip y el puerto, esa son las direcciones en internet.

Protocolos

Existen protocolos abiertos y cerrados, donde los cerrados solo los propietarios saben como funciona

open protocols:

- defined in RFCs, everyone has access to protocol definition
- allows for interoperability
- e.g., HTTP, SMTP

proprietary protocols:

- e.g., Skype, Zoom

Que necesitan los servicios?

- Necesitan integridad de datos(algunos aceptan perdida)
- Timing, necesitamos un delay bajo
- throughput, muchas apps necesitan poco, bits/s

- Seguridad, encriptacion de datos

<u>application</u>	<u>data loss</u>	<u>throughput</u>	<u>time sensitive?</u>
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video:10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

Protocolos de servicio

1. TCP

- Garantiza la integridad de los datos
- El enviador no puede saturar al receptor
- tiene control de congestion
- se requiere un proceso entre el cliente y el receptor
- no provee: timing, throughput ni seguridad

2. UDP

- No garantiza la integridad de datos
- no provee. integridad, control de flujo, control de congestión, timing, throughput, seguridad o conexión.

TCP service:

- *reliable transport* between sending and receiving process
- *flow control*: sender won't overwhelm receiver
- *congestion control*: throttle sender when network overloaded
- *connection-oriented*: setup required between client and server processes
- *does not provide*: timing, minimum throughput guarantee, security

UDP service:

- *unreliable data transfer* between sending and receiving process
- *does not provide*: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup.

Q: why bother? *Why* is there a UDP?

<u>application</u>	<u>application layer protocol</u>	<u>transport protocol</u>
file transfer/download	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP 1.1 [RFC 7320]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP or UDP
streaming audio/video	HTTP [RFC 7320], DASH	TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

Seguridad TCP

1. Vanilla TCP y UDP

- Esta todo en texto plano

- No es seguro

2. Transport Layer Security(TLS)

- Esta encriptado
- Integridad de datos
- Autenticacion por endpoint

Web y HTTP

Para acceder a una web primero ponemos el nombre del host y luego el path

www.someschool.edu / someDept/pic.gif

host name path name

Vista de HTTP

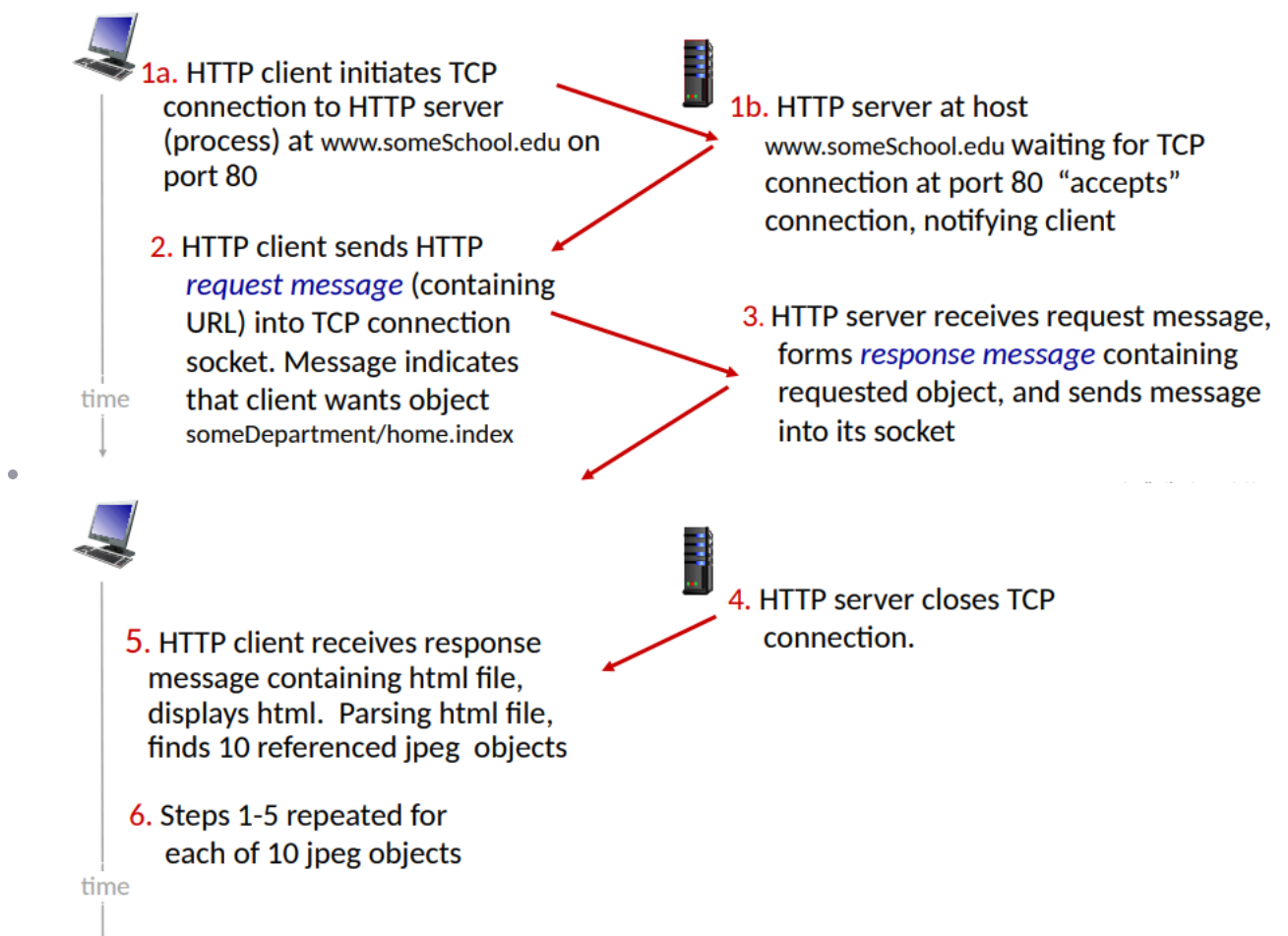
http(hypertext transfer protocol) es un protocolo, que el cliente recibe y permite al navegador desplegar la pagina y el servidor envía este protocolo.

http usa TCP, o sea el cliente manda una petición por el puerto 80, es recibido y se cierra la conexión, este protocolo no guarda la información del cliente anterior.

Conexiones HTTP: dos tipos:

1. No persistente HTTP

- Se abre la conexión
- se envía el objeto
- se cierra la conexión



2. HTTP persistente

- se abre la conexión TCP
- Muchos objetos se envían por este socket
- se cierra la conexión

HTTP no persistente

Palabra	Definicion
RTT	Tiempo en que un paquete pequeño viaja del cliente al servidor y de vuelta

HTTP response time(per object):

- Un RTT inicia la conexión TCP
- Se crea una respuesta
- Object/File transmisión

$$NonPersistentHTTPResponseTime = 2RTT + FileTransmissionTime$$

La conexiones no persistentes tienen unos problemas, requiere dos RTTs por objeto, el sistema crea overhead por cada conexión TCP y los navegadores ofrecen múltiples conexiones TCP de manera paralela, para referenciar objetos

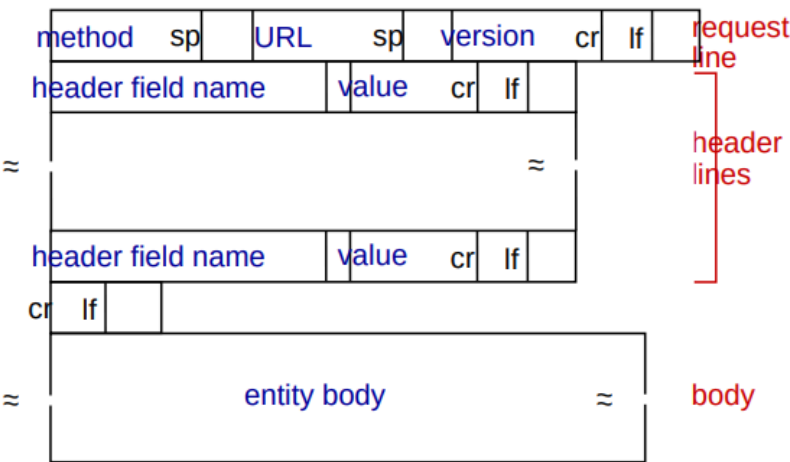
HTTP persistente (HTTP1.1)

Se mantiene la conexión abierta por lo que no requerimos de muchas llamadas RTT para cargar los objetos, si no que se abre la conexión y se mantiene abierta para el envío de objetos, logrando reducir el tiempo de carga

Request mensajes

se hace en ASCII.

Formato general



Estas son peticiones request:

POST method:

- web page often includes form input
- user input sent from client to server in entity body of HTTP POST request message

GET method (for sending data to server):

- include user data in URL field of HTTP GET request message (following a '?'):
`www.somesite.com/animalsearch?monkeys&banana`

HEAD method:

- requests headers (only) that would be returned if specified URL were requested with an HTTP GET method.

PUT method:

- uploads new file (object) to server
- completely replaces file that exists at specified URL with content in entity body of POST HTTP request message

y existen varios códigos de respuestas:

200 OK

- request succeeded, requested object later in this message

301 Moved Permanently

- requested object moved, new location specified later in this message (in Location: field)

400 Bad Request

- request msg not understood by server

404 Not Found

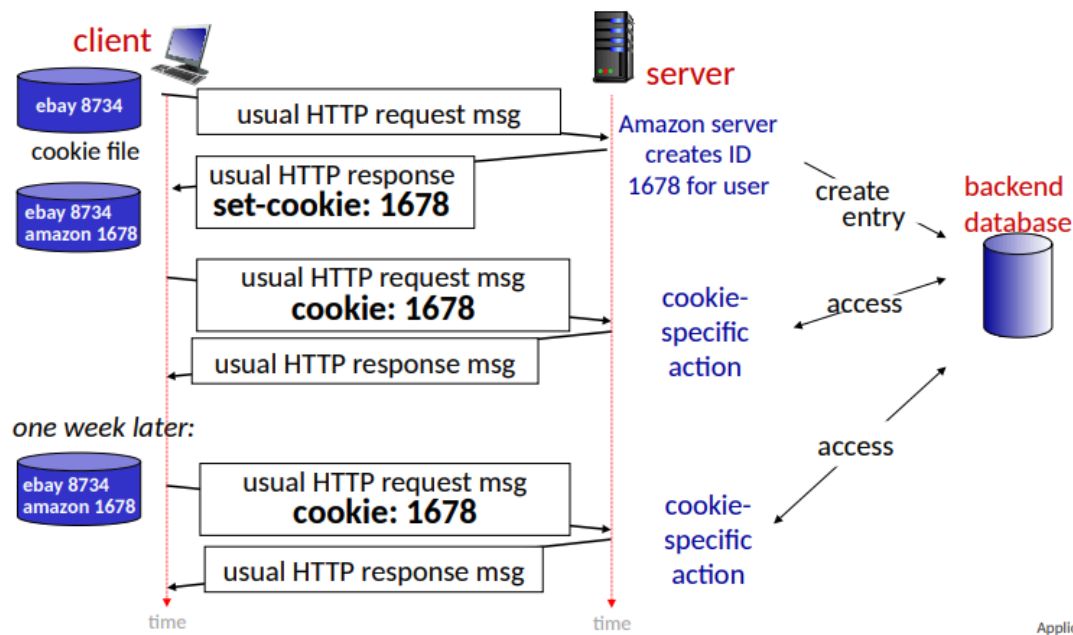
- requested document not found on this server

505 HTTP Version Not Supported

Application Layer: 2-34

Cookies

Muchos navegadores usan cookies para mantener el estado de una transacción, las cookies se mantienen en el host de usuario administrado por el navegador y también una parte se guarda en la base de datos del sitio web.



Application Layer: 2-34

aside

cookies and privacy:

- cookies permit sites to learn a lot about you on their site.
- third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites

Web Caches

La web cache es una copia del html que se guarda de manera local en el navegador, este hace una consulta en el servidor para ver si la versión guardada sigue estando actualizada, se devuelve una cache al cliente (una versión guardada de manera local de la página). También se conocen como proxy servers, actúa en ambos lados (cliente/servidor), el header dice como almacenar la cache

Cache-Control: max-age=<seconds>

Cache-Control: no-cache

Reduce el tiempo de petición, la cache esta mas cerca del cliente, reduce el trafico en los enlaces de instituciones y el Internet esta lleno de cache

Caching example

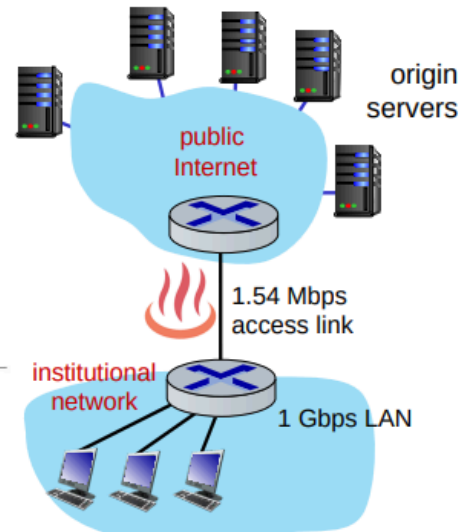
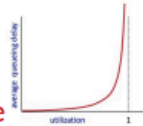
Scenario:

- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
 - avg data rate to browsers: 1.50 Mbps

Performance:

- access link utilization = .97
- LAN utilization: .0015
- end-end delay = Internet delay + access link delay + LAN delay = 2 sec + minutes + usecs

problem: large queueing delays at high utilization!

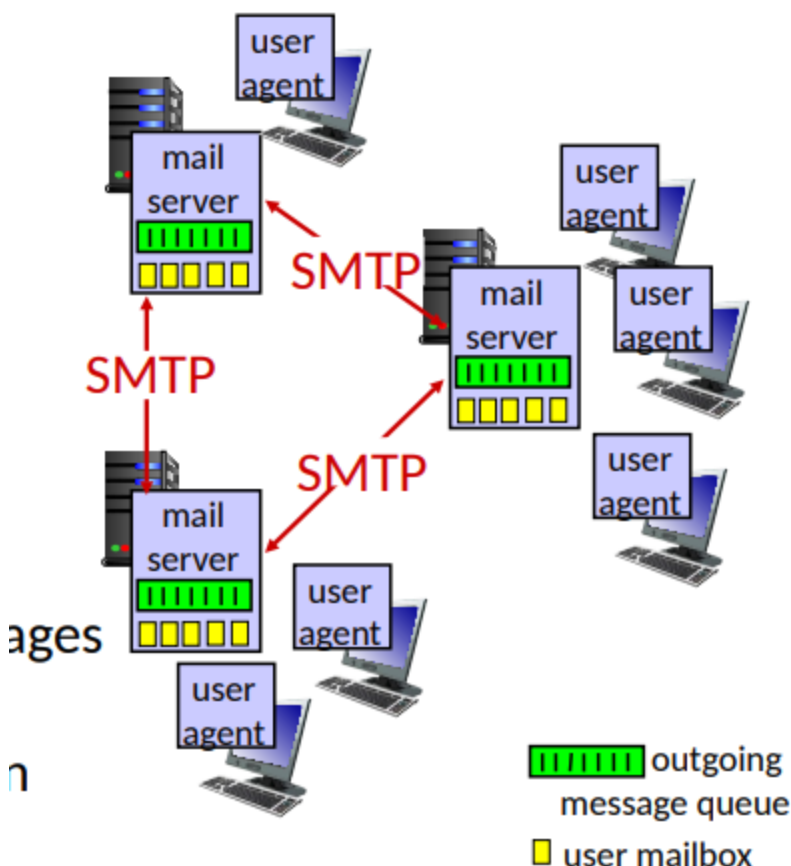


Application Layer: 1

En la actualidad las web caches no son del todo necesarias debido a que la mayoría de paginas son dinámicas(con backend) y que tenemos altas velocidades de red

Email, SMTP, IMAP

Email



Tiene 3 componentes mayores

- User agents

- Mail servers:
 - MailBox: Contiene los mensajes entrantes para el usuario
 - Message queue: cola de mensajes salientes que van a ser enviados
- Simple mail transfer protocol(SMTP):
 - Entre servidores de emails para enviar mensajes
 - Cliente: envía al servidor de mails
 - "Servidor": Recibiendo servidores de mails

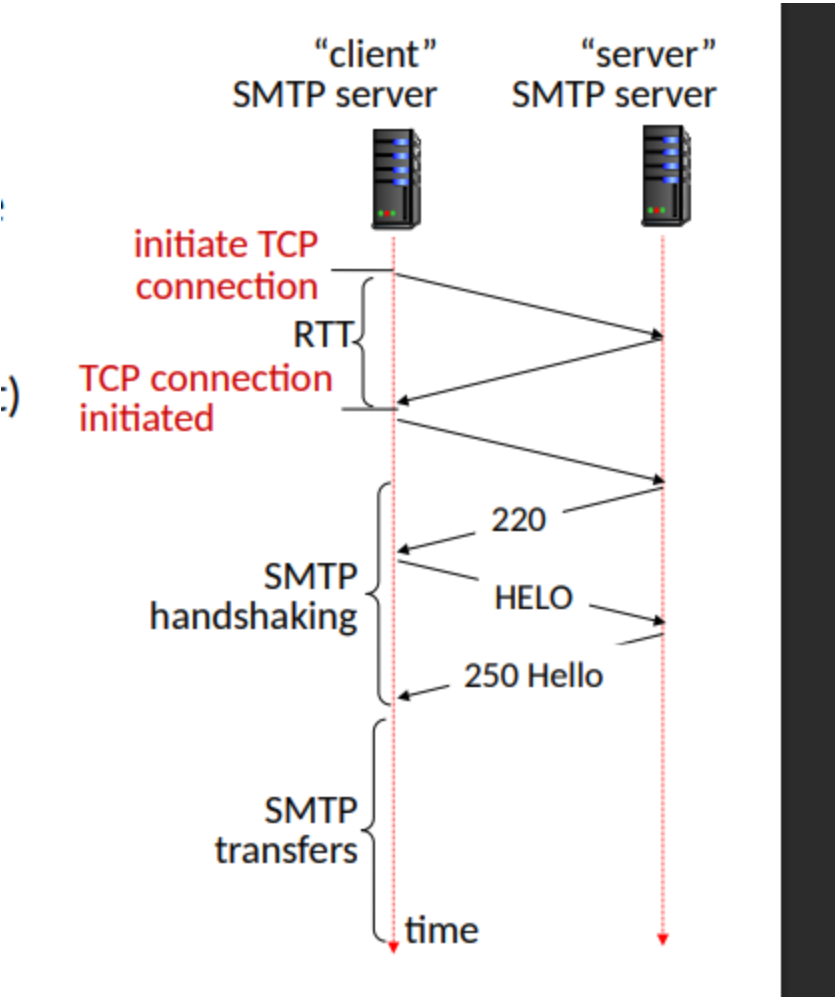
SMTP RFC(5321)

usa TCP para garantizar la correcta transacción de información entre datos, se usa el puerto 25

- Transferencia directa: Envía al servidor(actuando como cliente) para recibir al servidor
- Hay 3 fases de la transferencia:

1. SMTP Handshaking
2. SMTP transferencia de mensajes
3. SMTP cierre

Los comandos son en ASCII y las respuestas son con códigos y frases como en HTTP



comparison with HTTP:

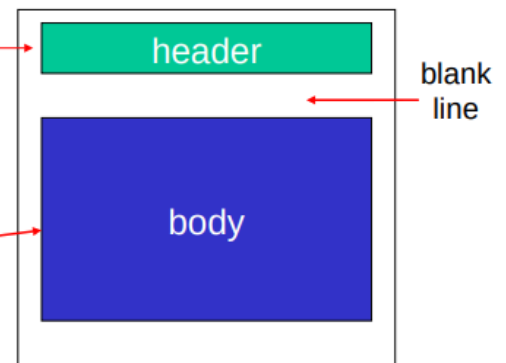
- HTTP: client pull
- SMTP: client push
- both have ASCII command/response interaction, status codes
- HTTP: each object encapsulated in its own response message
- SMTP: multiple objects sent in multipart message
- SMTP uses persistent connections
- SMTP requires message (header & body) to be in 7-bit ASCII
- SMTP server uses CRLF.CRLF to determine end of message

Mail message format

SMTP: protocol for exchanging e-mail messages, defined in RFC 5321 (like RFC 7231 defines HTTP)

RFC 2822 defines *syntax* for e-mail message itself (like HTML defines syntax for web documents)

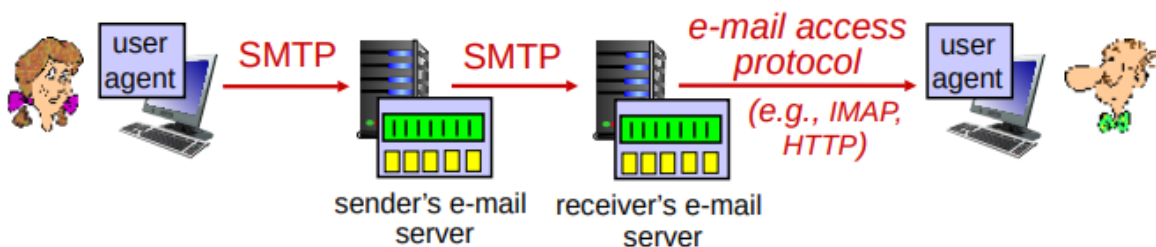
- header lines, e.g.,
 - To:
 - From:
 - Subject:these lines, within the body of the email message area different from SMTP MAIL FROM:, RCPT TO: commands!
- Body: the "message", ASCII characters only



Recuperando mails

El SMTP almacena y envía mensajes, es un servidor. Luego tenemos IMAP(Internet Mail Access Protocol) que nos permite acceder vía web a nuestros correos almacenados, podemos eliminar, almacenar,etc.

Gmail, Outlook, Hotmail, etc nos dan una interfaz web para acceder a nuestros correos.



DNS: Domain Name System

Nos permite acceder mas fácil a las paginas, ya que crea un enlace entre un nombre y una ip, de esta manera accedemos a usm.cl y no a 213.133.12.1.

Es una base de datos distribuida que esta implementada en las librerias de muchos servidores. Esta en la capa de aplicación: host, DNS resuelve nombres.

- Es el núcleo del Internet
- es complejo en los limites de la red

DNS: Servicios y estructura

- Hostnames a direcciones ip.
- Alias de host.

- Alias de servidores de mail.
- Distribución de carga.

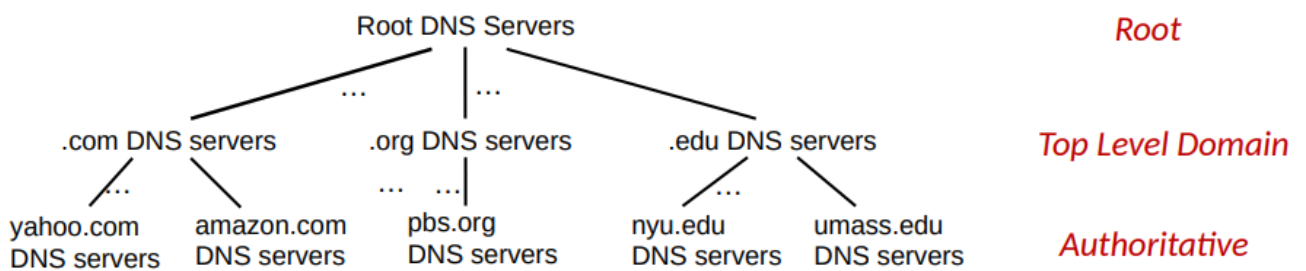
No podemos tener un DNS centralizado debido a que si falla se cae internet, tiene un volumen de trafico alto, una base de datos distante y el mantenimiento

Si pensamos en los DNS:

- Es una base de datos distribuidas y homogenia
- Maneja trillones de consultas
- Organizado pero físicamente descentralizado
- Aprueba de balas

DNS: distribuido y jerárquico

DNS: a distributed, hierarchical database



Client wants IP address for www.amazon.com; 1st approximation:

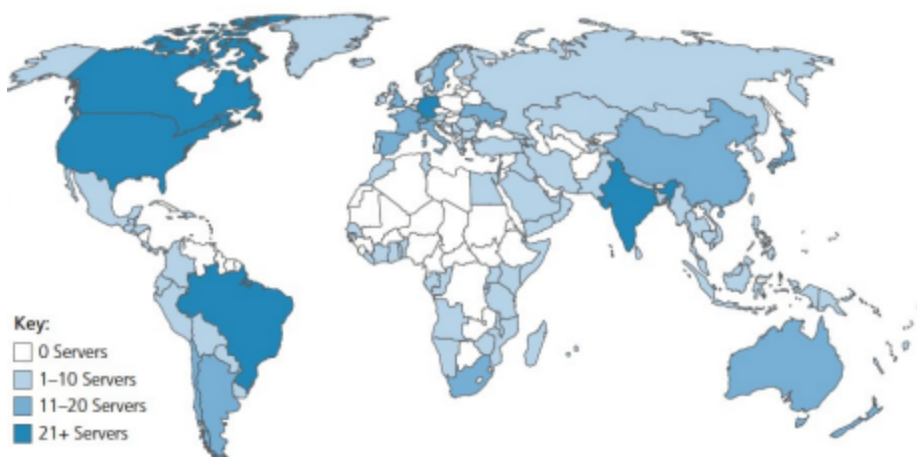
- client queries root server to find .com DNS server
- client queries .com DNS server to get amazon.com DNS server
- client queries amazon.com DNS server to get IP address for www.amazon.com

DNS: root name servers

Lo ideal es que a nivel top y autoritativo pueda resolver todas mis consultas, la ultima opción es consultar al root.

- El internet no funciona sin el
 - DNSSEC provee seguridad(autenticación e integridad de mensajes)
- ICANN(Internet Corporation for assigned Names and Numbers) Administra los root DNS

**13 logical root name “servers”
worldwide each “server” replicated
many times (~200 servers in US)**

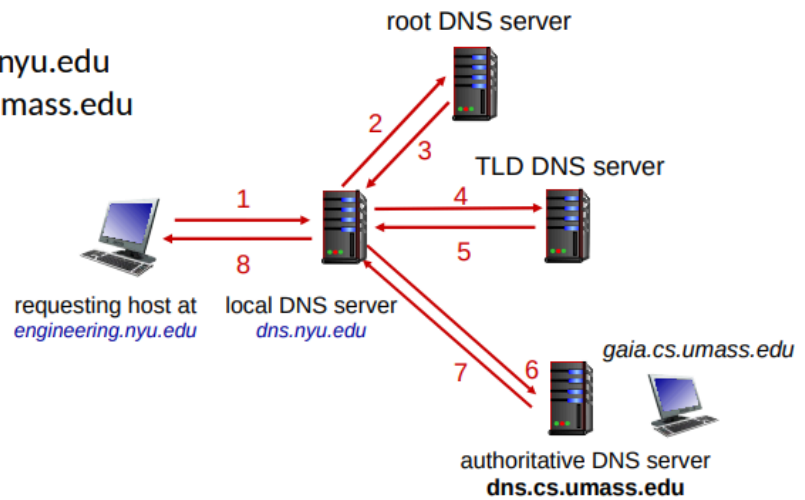


DNS name resolution: iterated query

Example: host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

Iterated query:

- contacted server replies with name of server to contact
- "I don't know this name, but ask this server"



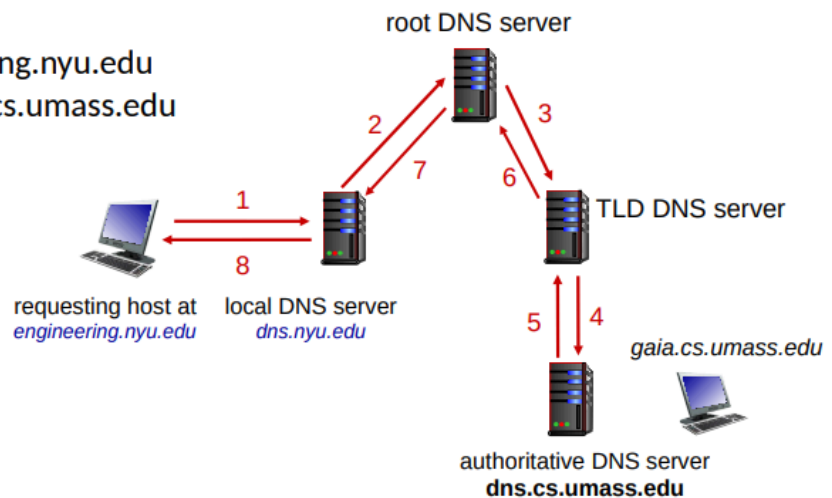
Los DNS locales suelen tener cache para acelerar la operación

DNS name resolution: recursive query

Example: host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



Caching DNS Information

- once (any) name server learns mapping, it *caches* mapping, and *immediately* returns a cached mapping in response to a query
 - caching improves response time
 - cache entries timeout (disappear) after some time (TTL)
 - TLD servers typically cached in local name servers
- cached entries may be *out-of-date*
 - if named host changes IP address, may not be known Internet-wide until all TTLs expire!
 - *best-effort name-to-address translation!*

DNS records

DNS: distributed database storing resource records (RR)

RR format: (name, value, type, ttl)

type=A

- name is hostname
- value is IP address

type=NS

- name is domain (e.g., foo.com)
- value is hostname of authoritative name server for this domain

type=CNAME

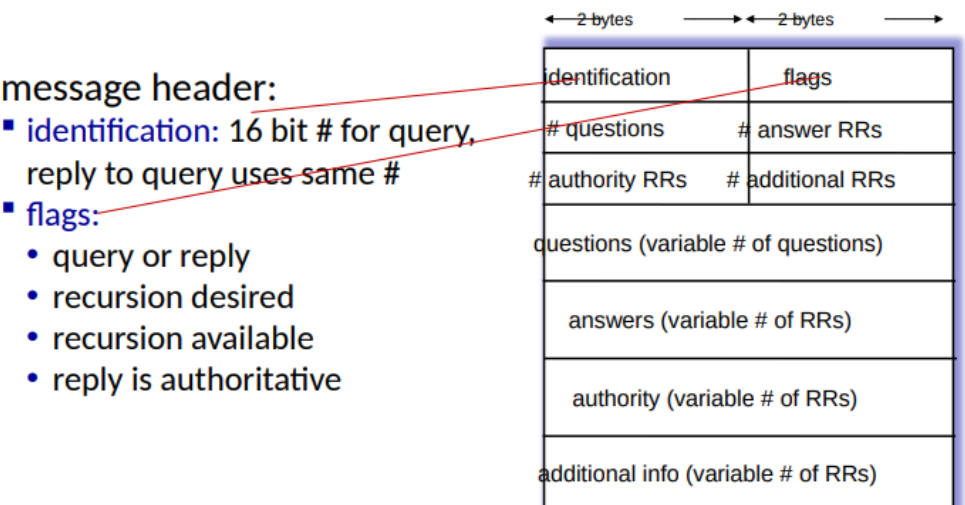
- name is alias name for some “canonical” (the real) name
- www.ibm.com is really servereast.backup2.ibm.com
- value is canonical name

type=MX

- value is name of SMTP mail server associated with name

DNS protocol messages

DNS *query* and *reply* messages, both have same *format*:



P2P aplicaciones

Arquitectura

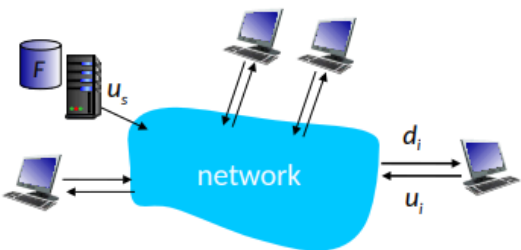
- No siempre hay servidores activos.
- Sistemas arbitrarios de comunicación.
- Un sistema solícito y el otro retorna.
- Hay un intercambio de direcciones de IP.
- Tiene auto escalabilidad.

server transmission: must sequentially send (upload) N file copies:

- time to send one copy: F/u_s
- time to send N copies: NF/u_s

client: each client must download file copy

- d_{min} = min client download rate
- min client download time: F/d_{min}



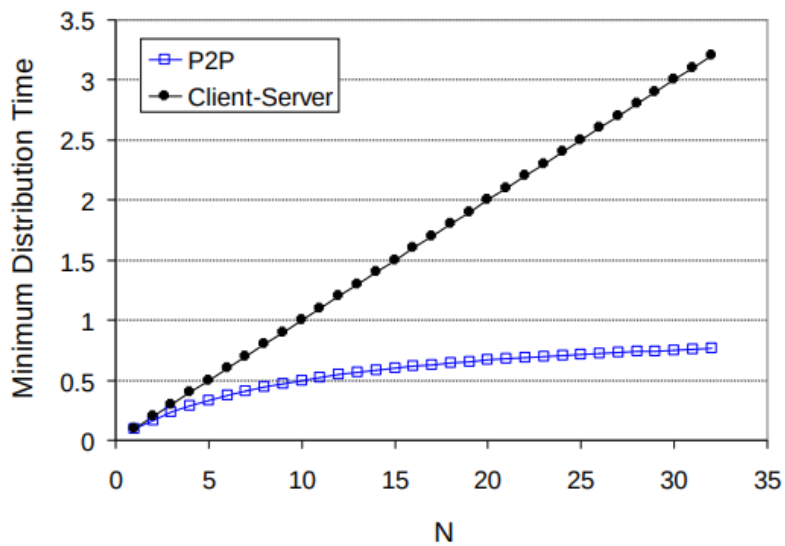
time to distribute F to N clients using client-server approach

$$T_{CS} > \max\{NF/u_s, F/d_{min}\}$$

increases linearly in N

El tiempo que demoran los archivos en descargarse es el tiempo mayor entre el "servidor" subiendo y el "cliente" descargando.

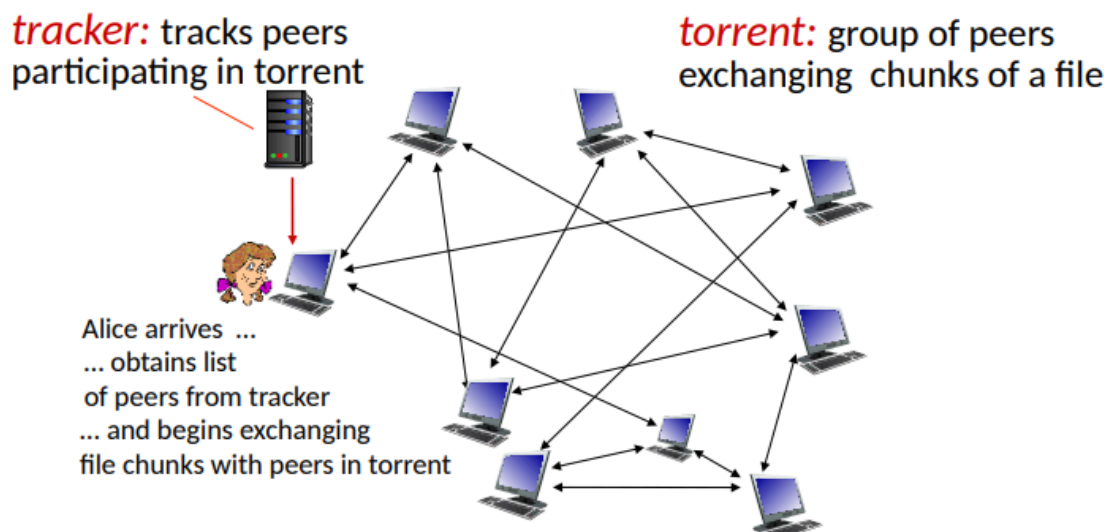
client upload rate = u , $F/u = 1$ hour, $u_s = 10u$, $d_{min} \geq u_s$



Mientras más clientes hay más óptimo es el modelo de P2P sobre cliente-servidor.

BitTorrent

- file divided into 256Kb chunks
- peers in torrent send/receive file chunks



la tecnología Torrent divide los archivos en chunks de 256kb, los peers reciben y envían chunks

- tracker: trackea los peers que participan en torrent.
- Torrent: grupo de peer que intercambian chunks del archivo.
Cuando uno entra a la red P2P no tiene chunks, pero se irán acumulando con el tiempo y con el tiempo uno también compartirá
- churn: peers que vienen y se van

Peticiones y envío de chunks de archivos

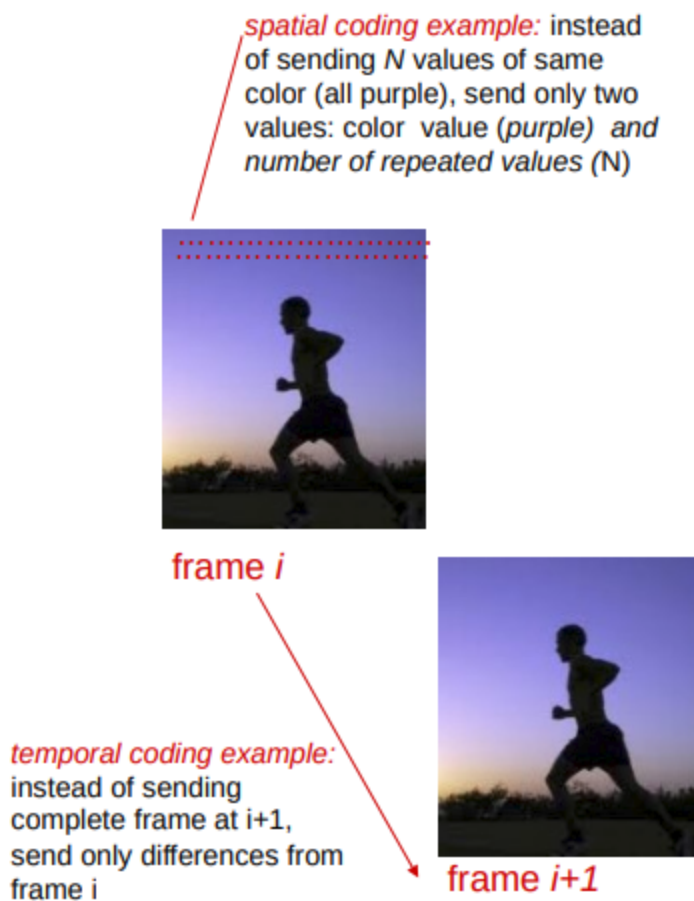
- Si hay carencia de ciertos chunks la red P2P automáticamente empezará a distribuir más estos chunks.
- Se usa un hash para todo el Torrent para verificar que no se está modificando él .torrent original, y además por cada chunk hay hash para verificar que no fue modificado.
- Cada 30 segundos, aleatoriamente se selecciona otro peer, este compuesto se llama tit-for-tat

Video streaming y distribución de contenido

- CDN(content distribution network)
- El streaming de video es lo que consume el mayor ancho de banda

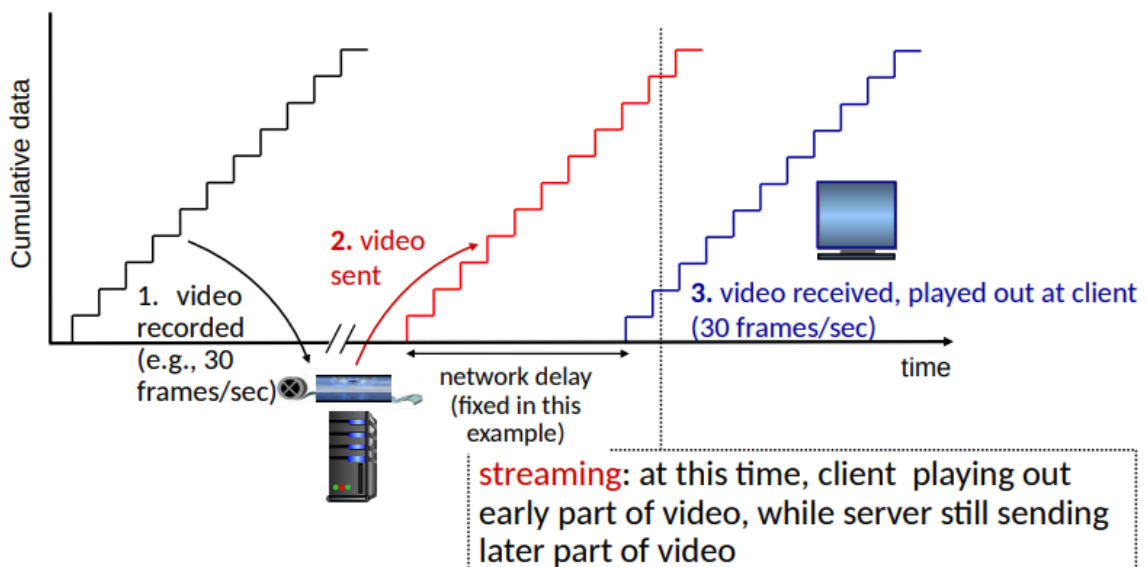
Video

- Los videos son secuencias de imágenes
- Cada imagen son un array de píxeles
 - cada pixel es representado por bits
- Coding: usa redundancia sin y entre imágenes para disminuir la cantidad de bits
 - Codificación espacial: Ve qué bits entre fotogramas no cambian y envían una instrucción para mantenerlos y de esa manera reducimos los bits que se envían
 - Codificación temporal: Solo envían los cambios que se realizan entre dos frames, para no recargar toda la imagen.



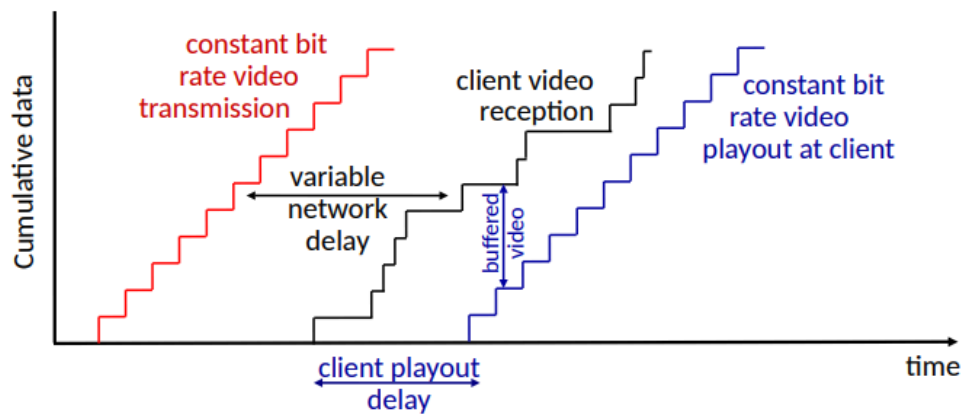
Codecs

- CBR(constant Bit Rate): siempre se gastan los mismos bits
- VBR(variable Bit Rate): con base en el movimiento de la imagen se gastan bits



Pero este es el mundo ideal, puesto que el internet no es constante

En la vida real existe el buffer que guarda bits de esa manera se compensa el delay



- **client-side buffering and playout delay:** compensate for network-added delay, delay jitter

Dash(Dynamic, Adaptive Streaming over HTTP)

1. Servidor

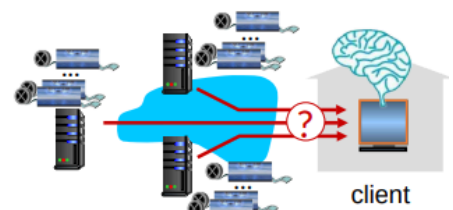
- Divide el video en chunks
- Cada chunks codifica en diferentes ratios
- diferentes rate de encoding se almacenan en distintos archivos
- los archivos se replican en varios nodos de CDN
- **Manifest file:** provee las direcciones de los diferentes chunks

2. Cliente

- Estima constantemente el ancho de banda entre cliente-servidor
- Consulta el manifiesto
- Elige un coding rate óptimo para su conexión

- **“intelligence” at client:** client determines

- **when** to request chunk (so that buffer starvation, or overflow does not occur)
- **what encoding rate** to request (higher quality when more bandwidth available)
- **where** to request chunk (can request from URL server that is “close” to client or has high available bandwidth)



Streaming video = encoding + DASH + playout buffering

CDN

- Realiza copias del contenido en los nodos CDN
- El suscriptor solicita contenido y el servicio provee el manifiesto y la dirección cambiara en base al ancho de banda

CH3: capa de transporte

Servicio de la capa de transporte

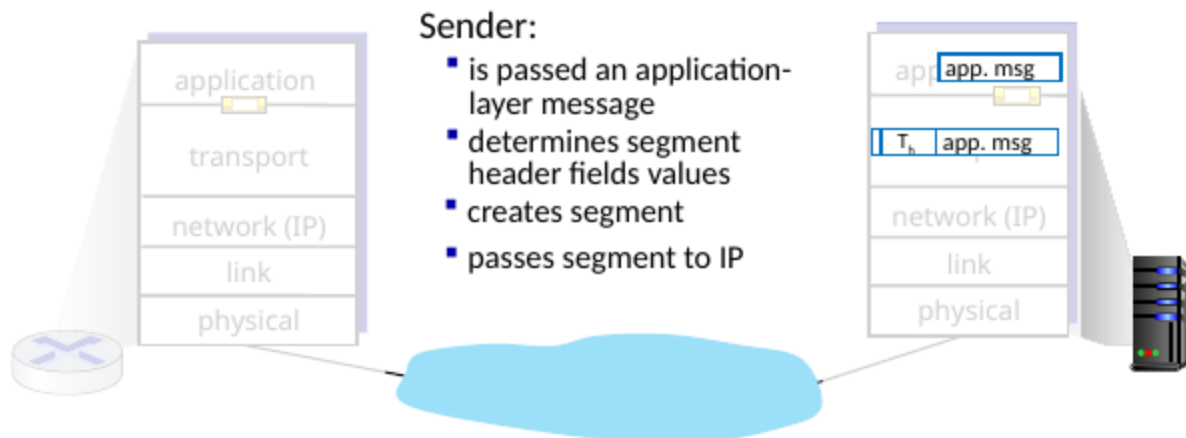
Provee comunicación lógica entre aplicaciones, funciona de la siguiente manera

- El emisor: rompe el mensaje en segmentos que pasan por la capa de red
- El receptor: reensambla los segmentos para obtener el mensaje

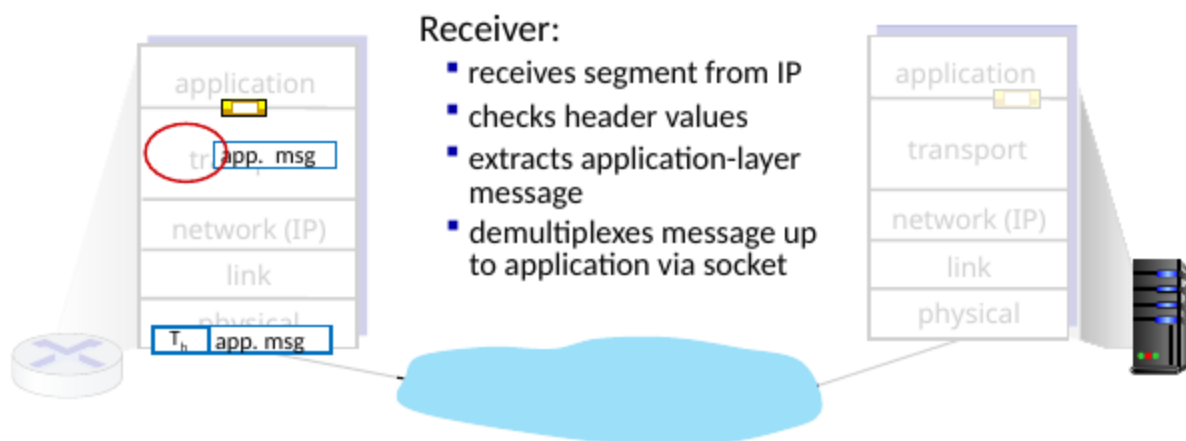
existen 2 protocolos **TCP** y **UDP**

La acción sería más o menos a si:

Transport Layer Actions



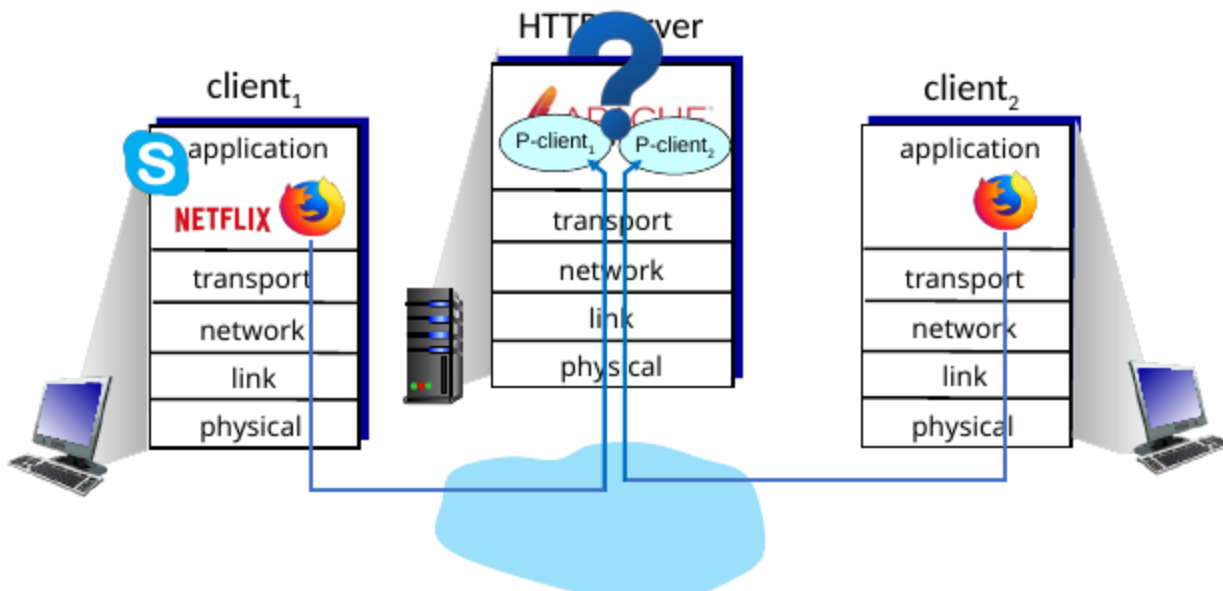
Transport Layer Actions



Los dos principales protocolos de internet

- **TCP(Transmission Control Protocol):**
 - Entrega en orden y es confiable
 - Control de congestion
 - Control de flujo
 - Setup de conexion
- **UDP(User Datagram Protocol):**
 - no es confiable, entrega desordenado
 - Una extensión basica del protocolo IP, e base al mejor esfuerzo
- **Servicio no disponible:*
 - Garantiza delay
 - Garantiza ancho de banda

Multiplexación y desmultiplexación

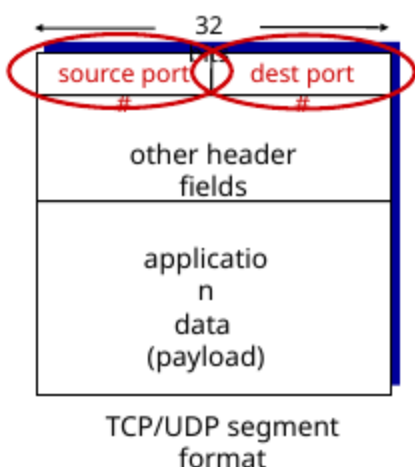


La multiplexación es una manera de juntar datos para su envío, imagínate que en un celular tenemos muchas apps ejecutándose, cada una enviando información, pero solo disponemos de un cable para enviar información, entonces el computador realiza una multiplexación en la cual realiza lo siguiente:

- Recoge datos de múltiples sockets
 - Les agrega header
 - Y mete segmentos
- Los datos se envían y listo, pero por ejemplo un servidor que recibe peticiones hace el proceso inverso la demultiplexación:
- Lee los header
 - Identifica que segmento le pertenece a que socket
 - Entrega la app correcta

Como funciona la demultiplexación

- Cada host recibe el datagrama del IP
 - Cada datagrama tiene como origen una IP y un destino que también es una IP
 - Cada datagrama lleva un segmento de la capa de transporte
 - Cada segmento tiene un origen y un puerto de destino
- Los hosts usan direcciones IP y puertos para que el segmento llegue al socket correcto



Demultiplexación sin conexión

- Cuando creamos un socket debemos especificar el puerto del host local
- Cuando creamos un datagrama que envía en UDP socket, debemos especificar
 - IP de destino
 - Puerto de destino

- ## Connectionless demultiplexing: an example

Conexión orientada a desmultiplexación

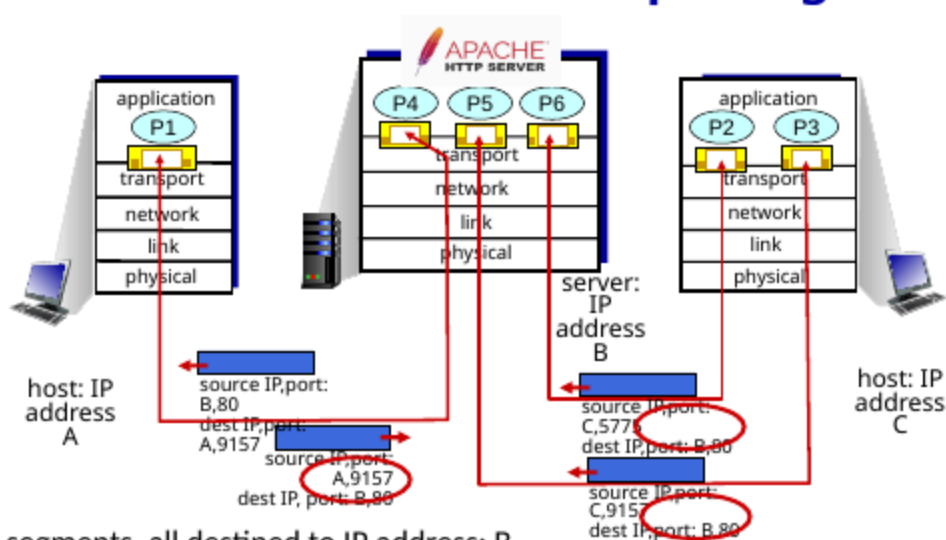
Los **TCP Socket** se identifican por 4 tuplas

- IP de origen
- Puerto de origen
- IP de destino
- Puerto de destino

El demux recibe los 4 valores y los dirige al socket correcto.

Los servidores pueden soportar simultáneos sockets TCP, cada socket identificado por esas 4 tuplas y cada socket asociado a distintos clientes

Connection-oriented demultiplexing: example



Three segments, all destined to IP address: B,
dest port: 80 are demultiplexed to **different** sockets

La multiplexación y demultiplexación ocurre en todas las capas
Porque en cada capa:

- hay **múltiples “fuentes” de datos**
- se **combinan en una sola transmisión**
- y luego se **separan usando algún identificador**

Lo único que cambia es **qué campo se usa para separar**:

- Aplicación → procesos
- Transporte → puertos
- Red → protocolo (TCP/UDP)
- Enlace → tipo (EtherType)

Transporte no orientado a conexión: UDP

UDP

- Un protocolo sin nada
- El mejor esfuerzo
 - Se pierde.
 - Se puede entregar en distinto orden.
- Sin conexión
 - NO hay handshake entre el emisor y el receptor
- Pero tiene una utilidad, es más rápido, tiene un header pequeño, no hay RTT delay, es simple, no hay control de congestión por lo que usara todo el ancho de banda y puede funcionar con congestión

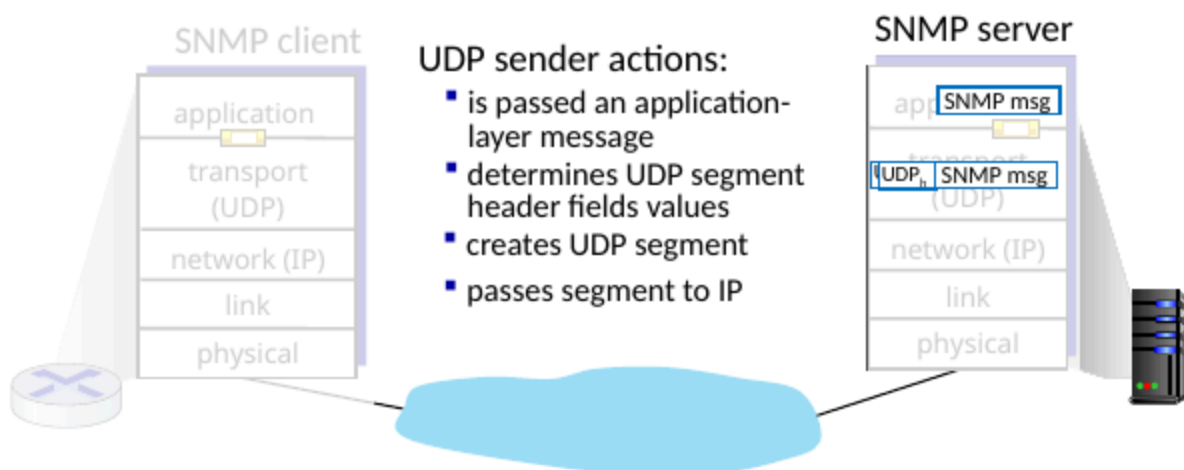
Cosas que usan UDP:

- Streaming multimedia
- DNS
- SNMP
- HTTP/3

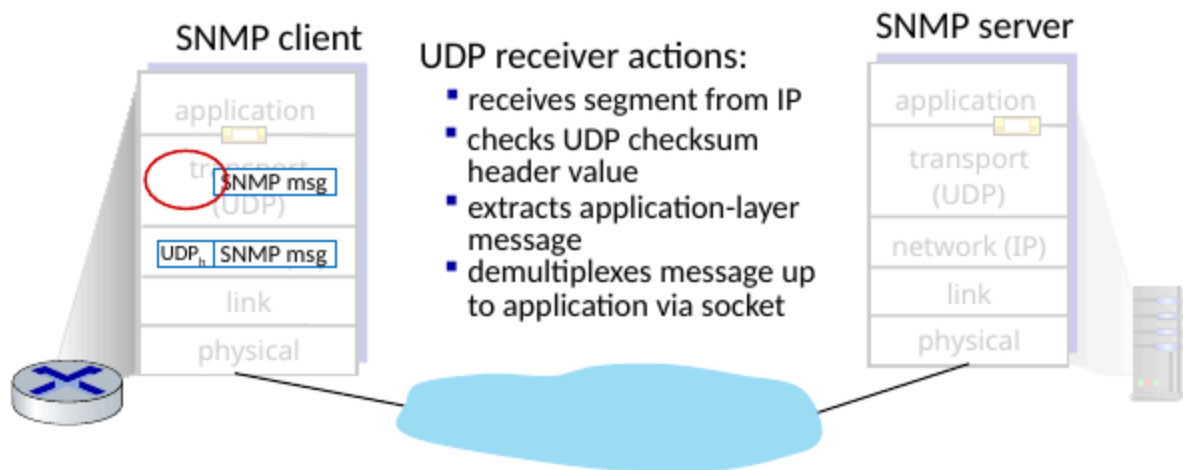
Para poder mejorar UDP tenemos que hacerlo desde la aplicación(HTTP/3):

- añadir confiabilidad
- añadir control de congestion

UDP: Transport Layer Actions

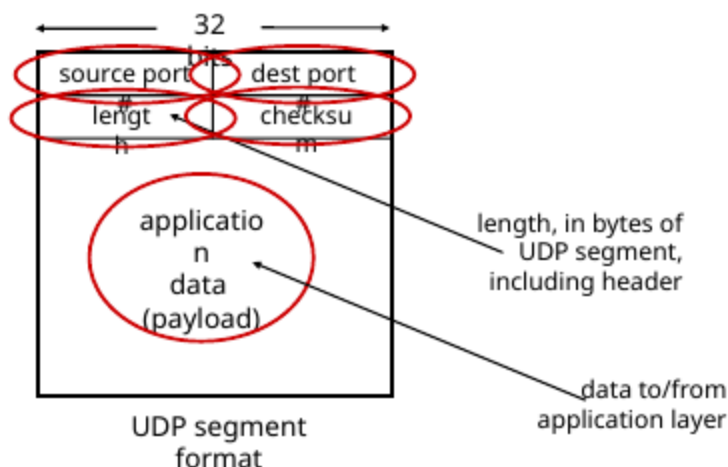


UDP: Transport Layer Actions



Header del segmento UDP

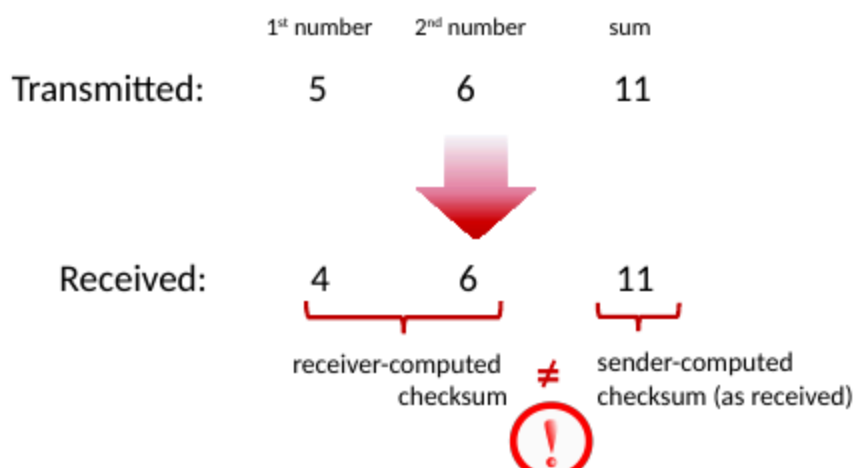
UDP segment header



UDP Checksum

UDP checksum

Goal: detect errors (i.e., flipped bits) in transmitted segment



el objetivo de esto es detectar errores

- Emisor
 - Trata el contenido en segmentos UDP como secuencia de integrales de 16 bits
 - Checksum: se añade al segmento del contenido

- El checksum se pone en el campo del checksum
- Receptor
 - El computador recibe el segmento con el checksum
 - checa que el checksum recibido es el mismo que el calculado
 - Si no es igual, hay error
 - Si es igual, no hay errores, de momento

Internet checksum: an example

example: add two 16-bit integers

	1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0
	1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
wraparound	1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1
sum	1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 0
checksum	0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1

Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Internet checksum: weak protection!

example: add two 16-bit integers

	1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0	0 1
	1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1	1 0
wraparound	1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1	
sum	1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 0	
checksum	0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1	

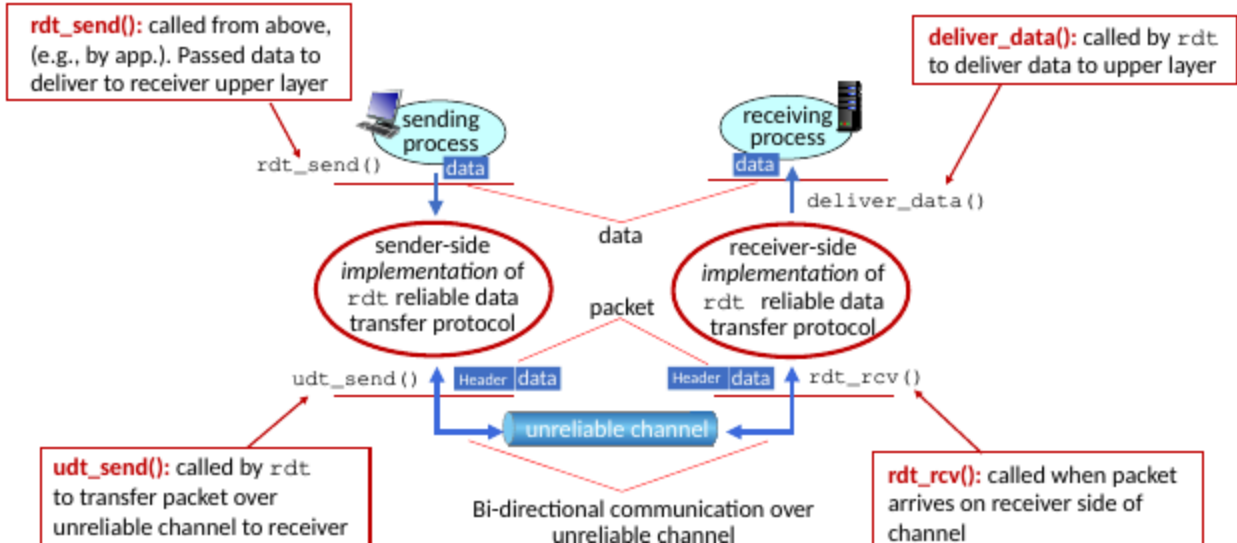
Even though numbers have changed (bit flips), *no* change in checksum!

Principio de la confiabilidad de la transferencia de datos

La complejidad de la confiablilidad de la data depende de un canal no confiable, esto puede provocar muchos problemas y para mas complejidad, el emisor y el receptor no conocen el estado del otro, por eso nace:

Reliable data transfer protocol (rtd): interfaces

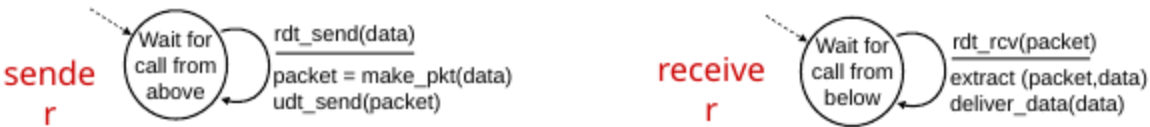
Reliable data transfer protocol (rdt): interfaces



En la vida real esta comunicaci3n funciona en ambos sentidos

rdt1.0: transferencia confiable sobre un canal no confiable

Vamos a usar una maquina de estado finito entre el emisor y el receptor



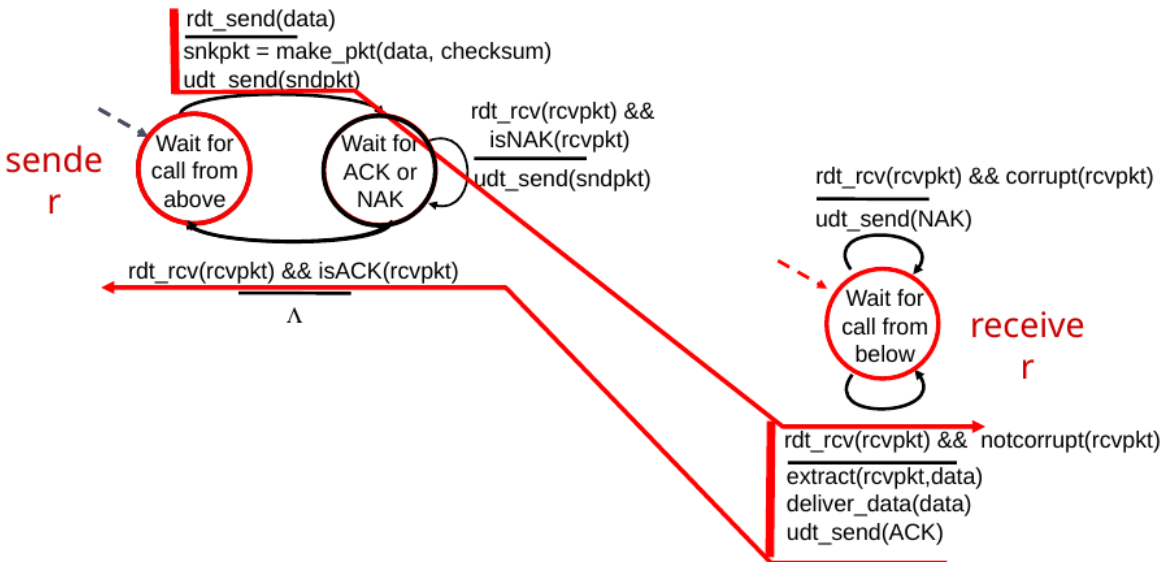
En este formato imaginamos que no hay perdida de paquetes y no hay errores de bits, por lo que se envía la data de un emisor a un receptor

rtd2.0: canal con errores de bits

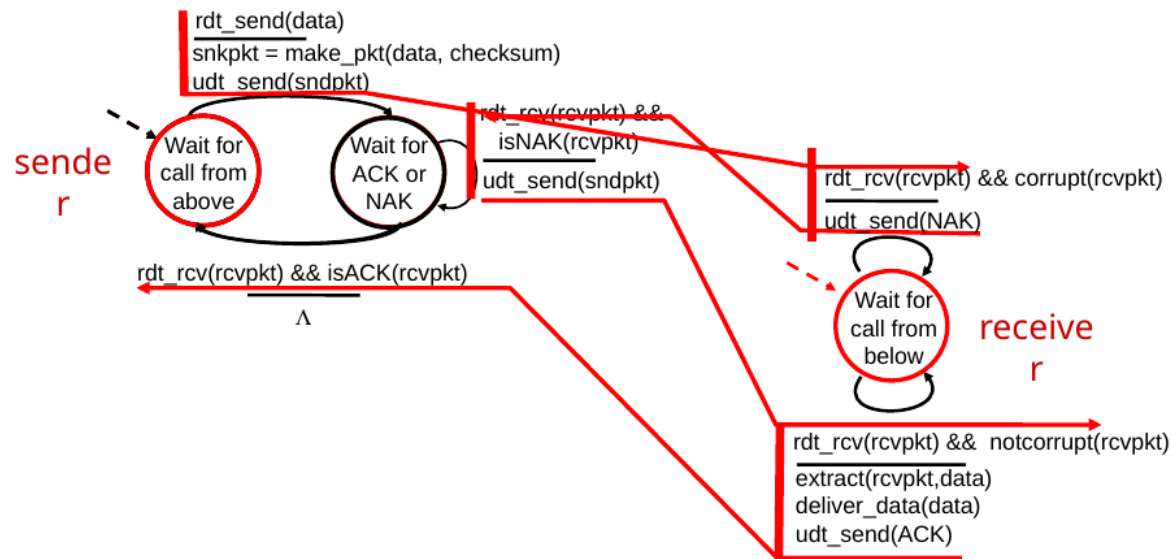
imaginemos que un bit se da vuelta ya sea por interferencia en el cable, etc. Para recuperarlo usamos

- acknowledgements(ACKs): el emisor explícitamente envía un OK
 - negative acknowledgements(NAKs): el emisor envía un mensaje de que se envi3 mal
- Entonces el emisor reenvía el mensaje

Sin errores



Con errores

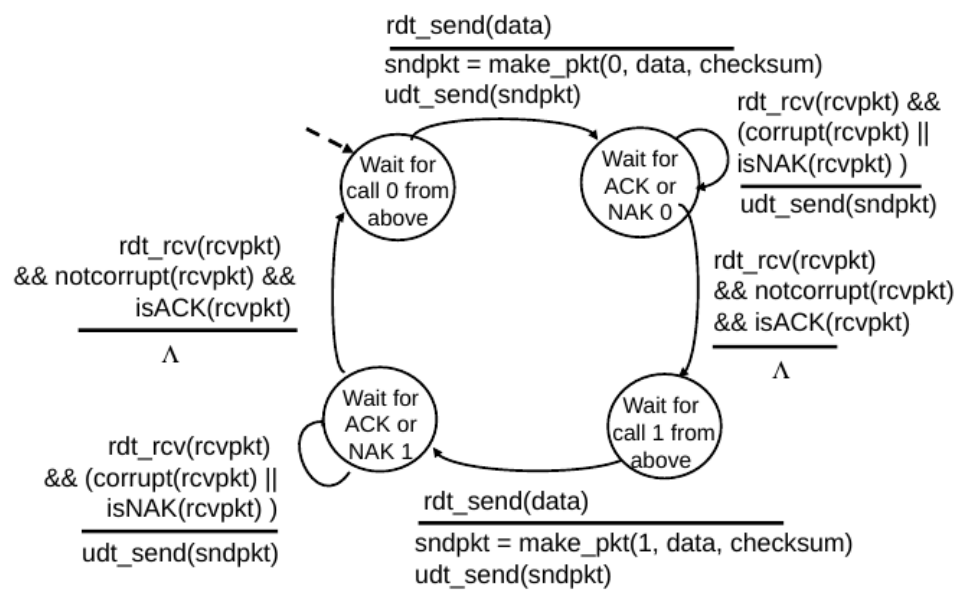


Debilidad de este modelo

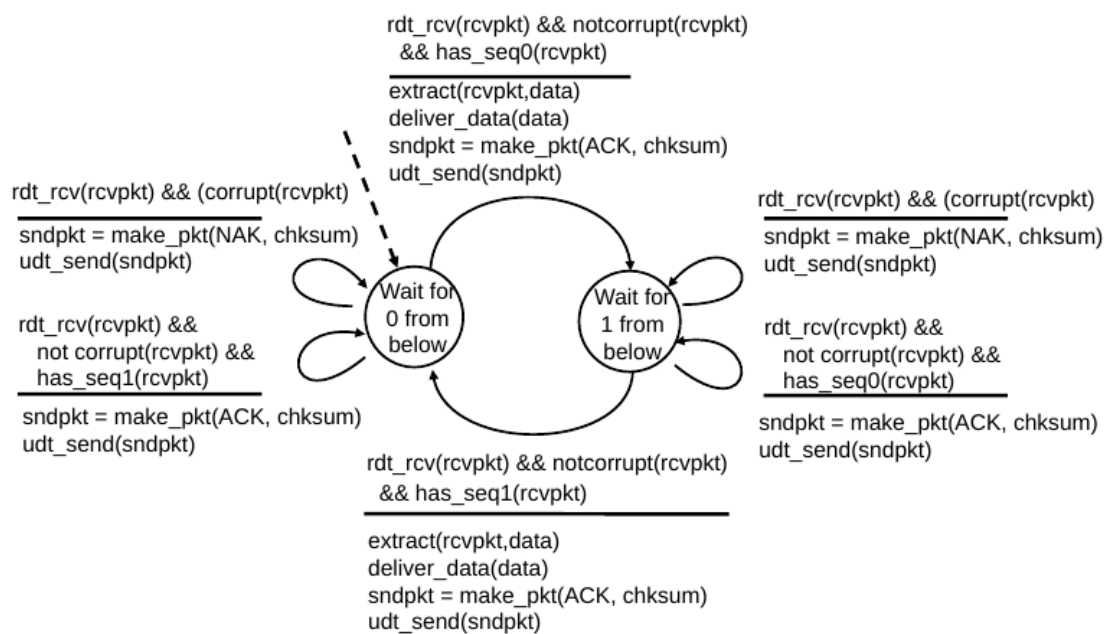
Se puede corromper el ACK/NAK y como sabemos el emisor no sabe lo que pasa con el receptor por lo que se puede enviar data duplicada, cada emisor envía un pkt por cada respuesta a la cual le agrega un número lo cual puede duplicar pkt(ACK/NAK)

rtd2.1

rdt2.1: sender, handling garbled ACK/NAKs



rdt2.1: receiver, handling garbled ACK/NAKs



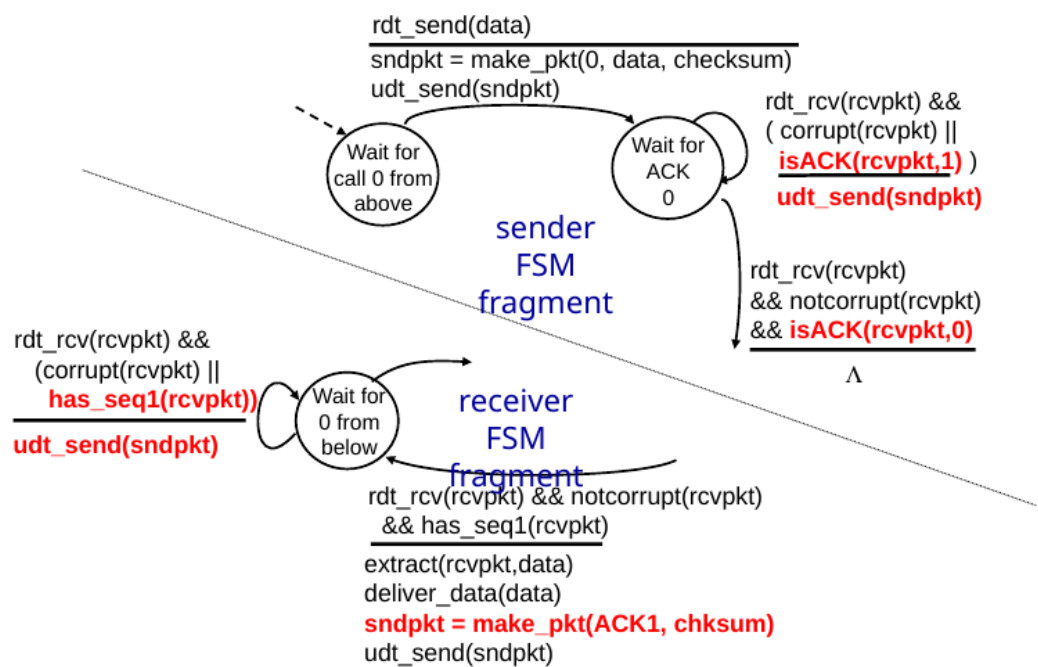
El emisor envía un pkt en secuencias de 0 y 1, entonces el emisor verifica este número para ver si está repetido.

rtd2.2 un protocolo libre de NAK

- Es lo mismo que rtd2.1 pero libre de NAK
- El receptor envía un ACK, debe incluir explícitamente la secuencia de pkt

- Los ACK duplicados son lo mismo que un NAK

rat2.2: sender, receiver fragments

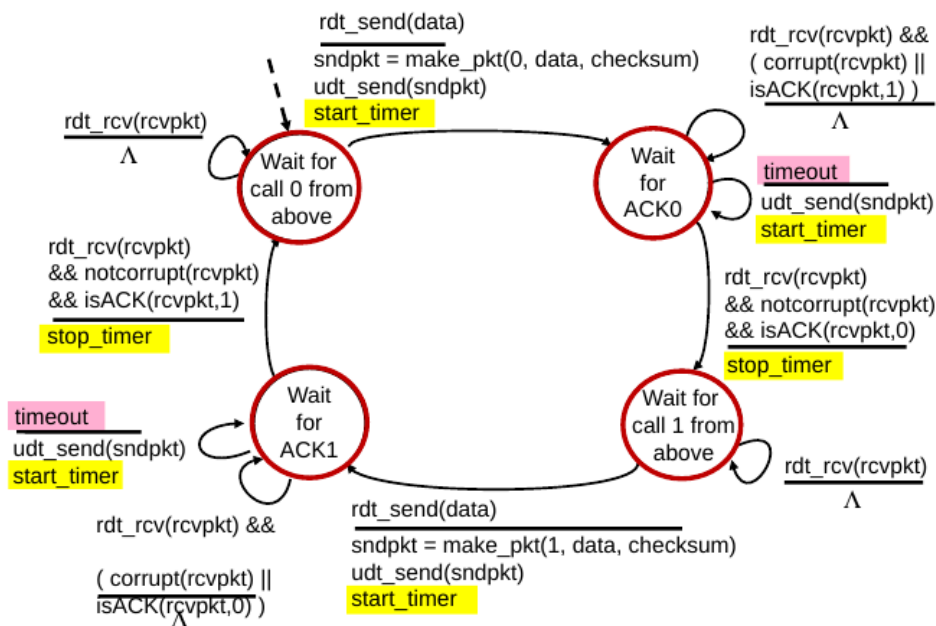


Transport Layer

rtd3.0 canal con error y perdida

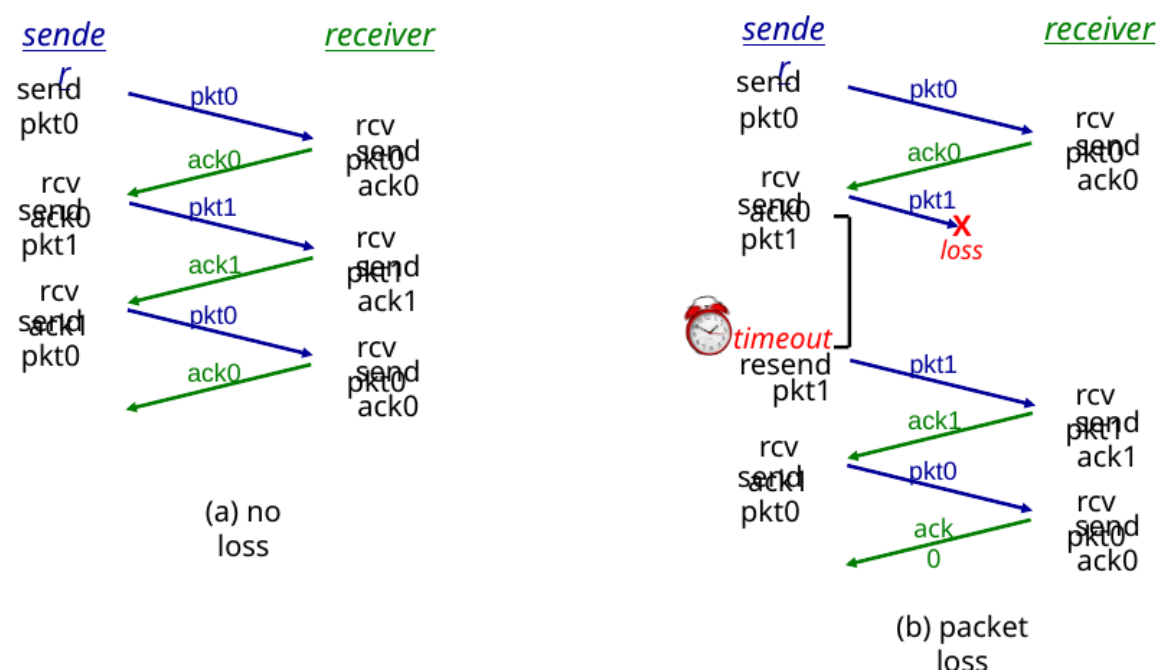
El emisor espera un tiempo razonable para esperar el ACK, si no responde un ACK, el emisor reenvía el paquete, pero la secuencia ya maneja duplicados, el receptor debe especificar el número de secuencia

rat3.0 sender



Transport Layer

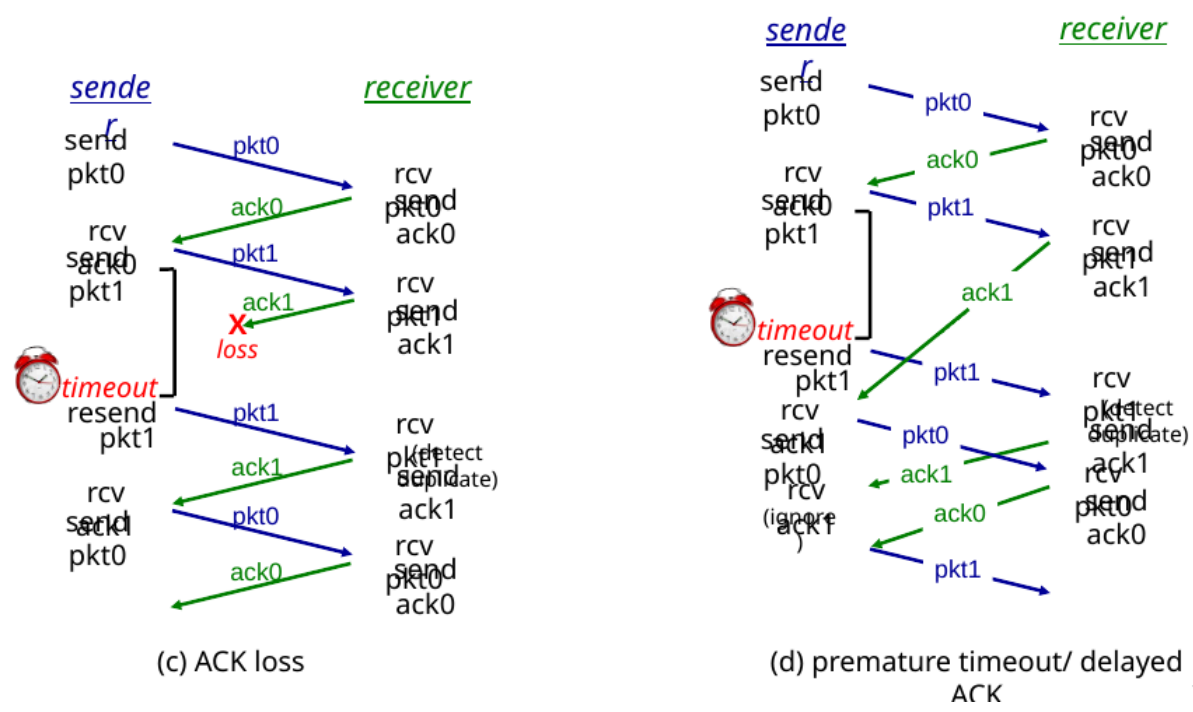
rdt3.0 in action



Transport Layer

Como vemos si el ACK se pierde el emisor lo vuelve a enviar

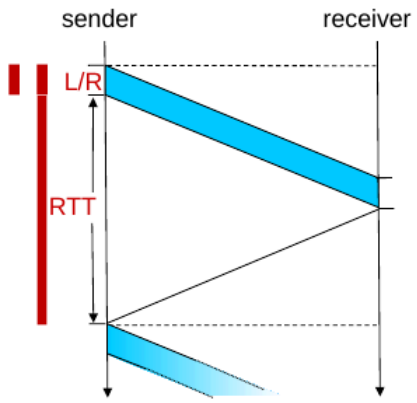
rdt3.0 in action



Transport Layer

rdt3.0: stop-and-wait operation

$$\begin{aligned} U_{\text{sender}} &= \frac{L / R}{RTT + L / R} \\ &= \frac{.008}{30.008} \\ &= 0.00027 \end{aligned}$$

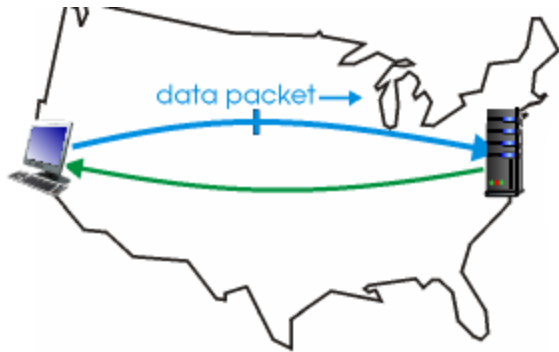


- rdt 3.0 protocol performance stinks!
- Protocol limits performance of underlying infrastructure (channel)

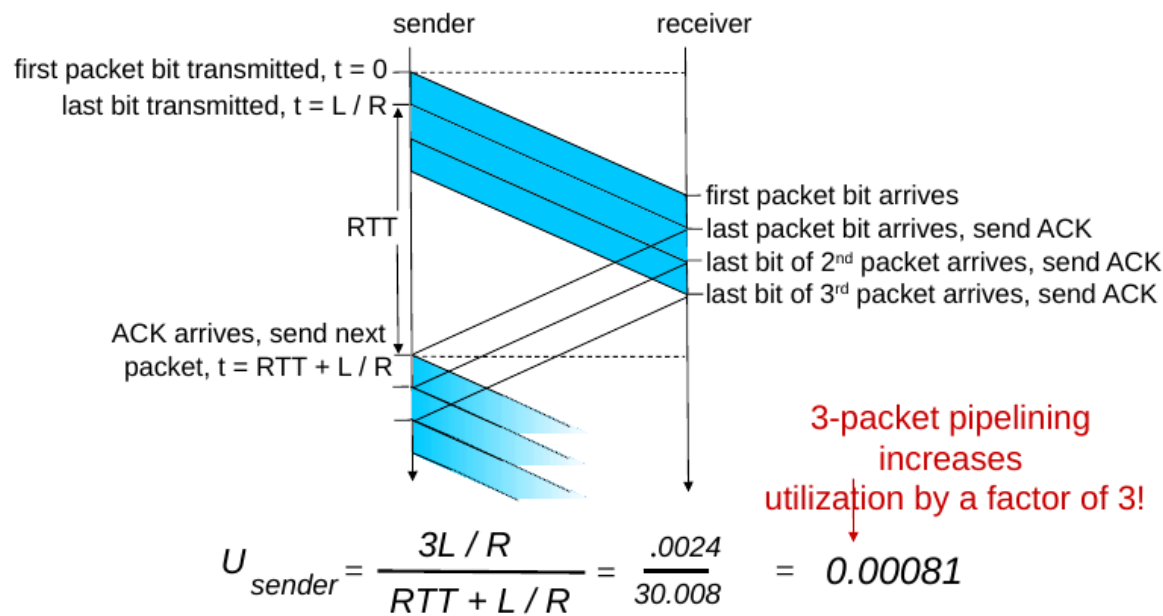
Transport Layer

El protocolo es muy lento, por lo que se usa pipelining, en donde el emisor envía muchos paquetes los cuales están en espera de confirmación, el rango de secuencia se va a

incrementar, hay un buffer entre el emisor y receptor

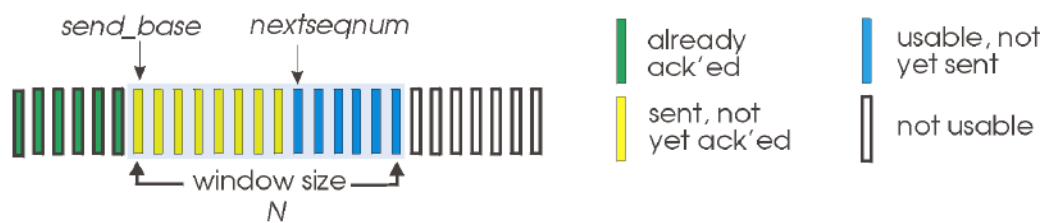


(a) a stop-and-wait protocol in operation



Go-Back-N: emisor

- Una ventana es la cantidad de paquetes que están buenos
 - k-bit secuencia en el header del paquete

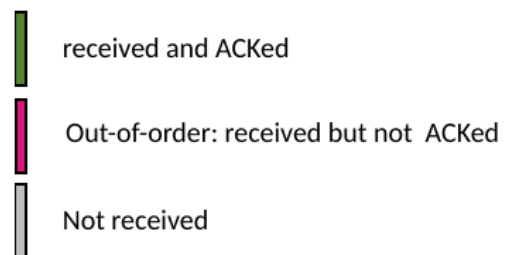
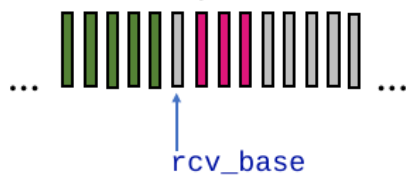


- **ACK acumulativos:** son todos los paquetes incluidos en la secuencia, cuando se recibe un ACK la ventan se mueve
- el timer es para el paquete en vuelo más viejo
- al haber timeout se envia el paquete n y todos los siguientes.

Go-Back-N: receptor

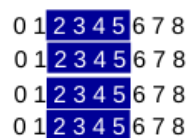
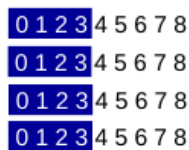
El receptor solo recibe paquetes en orden, si no lo están se ignoran o lo guarda en el buffer, el receptor solo debe recordar el último paquete correcto que se envió

Receiver view of sequence number space:

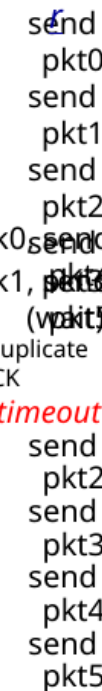


Go-Back-N in action

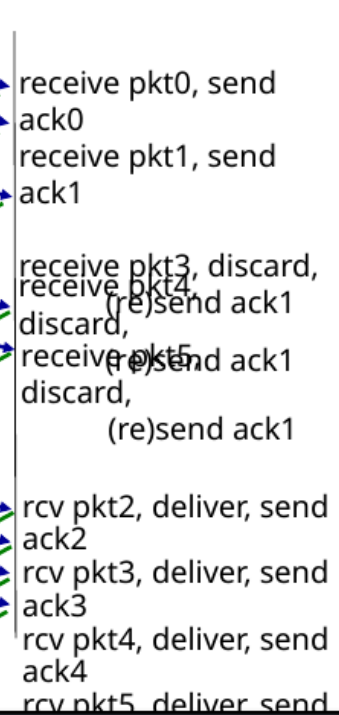
sender window (N=4)



sender



receiver

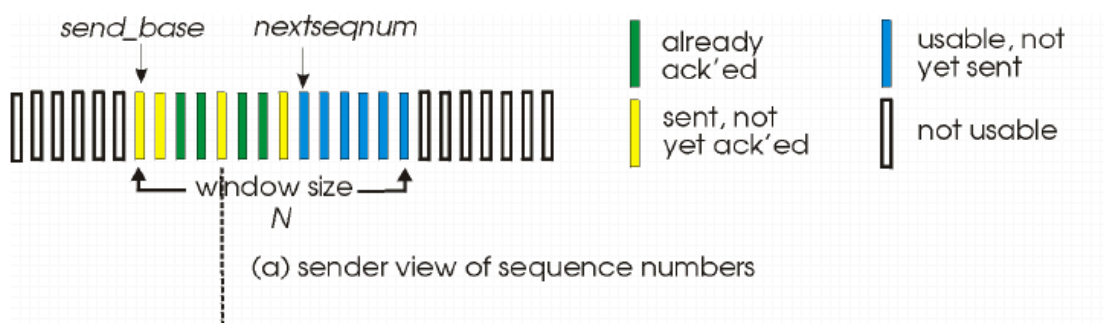


Trans

Selective repeat

Este es un tanto diferente al Go-Back-N, puesto que en este solo se reenvía el paquete faltante o dañado, por ejemplo si el emisor envía los paquetes {p1, p2 y p3} y el receptor recibe {p1 y p3}, confirma estos con un ACK y si el emisor no recibe un ACK en un tiempo dado por un paquete lo reenvía, como al emisor le falta un paquete intermedio guarda el p3 en el buffer esperando que llegue el p2, hay un número máximo de paquetes que pueden ser enviados antes de confirmar su recepción.

Selective repeat: sender, receiver windows



Parte 1: EMISOR (sender)

*Cuando llegan datos de arriba:

- Si el número de secuencia está dentro de la ventana → **envía el paquete**
- No espera ACK inmediato → puede mandar varios seguidos

Si hay timeout:

- Si el paquete **n no fue confirmado**:
 - Lo vuelve a enviar
 - Reinicia su temporizador

clave: **solo reenvía ese paquete**, no todos

Cuando recibe un ACK:

- Marca el paquete como **recibido**
- Si ese paquete era el más antiguo sin confirmar:
 - **mueve la ventana hacia adelante**

Ejemplo:

- Enviados: 1,2,3,4
 - Llega ACK de 1 → la ventana avanza
 - Llega ACK de 2 → sigue avanzando
-

Parte 2: RECEPTOR (receiver)

Cuando llega un paquete dentro de la ventana:

- Envía **ACK(n)**
 - Si está fuera de orden:
 - lo guarda (buffer)
 - Si está en orden:
 - lo entrega a la capa superior
 - entrega también los que tenía guardados en orden
-

Si llega un paquete “viejo”:

- Ya lo había recibido antes
- Igual manda ACK(n) otra vez

esto ayuda si el ACK original se perdió

Si está fuera de todo rango:

- Lo ignora completamente
-

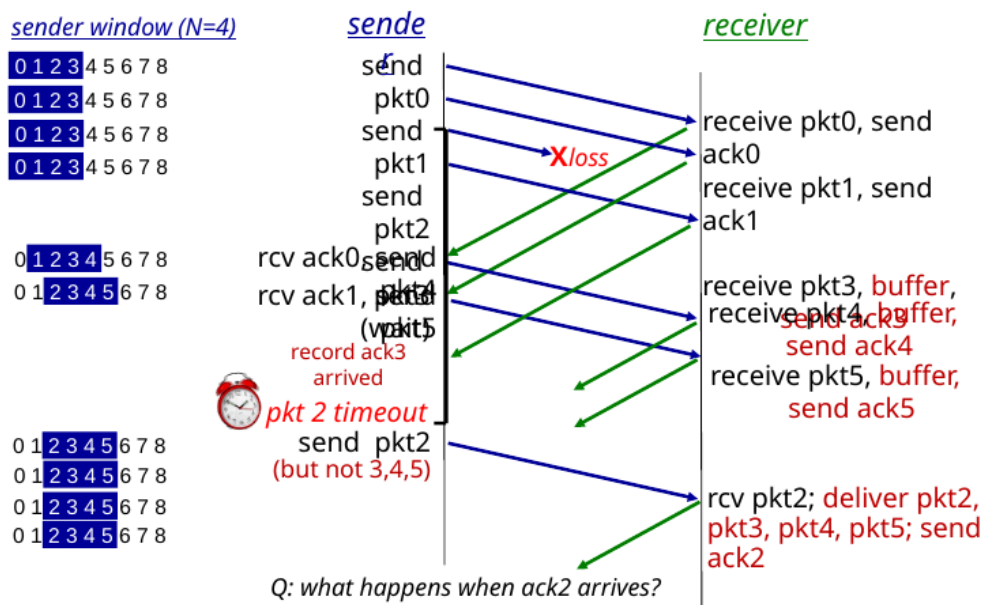
La clave de todo esto

Selective Repeat funciona porque:

- El receptor **acepta paquetes desordenados**

- El emisor **maneja cada paquete individualmente**
- Ambos usan **ventanas** para limitar el flujo

Selective Repeat in action



Intuición (lo importante de verdad)

Si la ventana es muy grande:

- Se “superpone” con números antiguos
 - El receptor **no puede distinguir pasado vs presente**
- Si la ventana es pequeña:

- Nunca confunde paquetes viejos con nuevos

Resumen corto

- Selective Repeat reutiliza números → peligro
- El receptor puede aceptar paquetes viejos por error

- Para evitarlo:
Ventana $\leq$ mitad del espacio de secuencia

Selective repeat: a dilemma!

example:

- seq #s: 0, 1, 2, 3 (base 4 counting)
- window size=3

Q: what relationship is needed between sequence # size and window size to avoid problem in scenario (b)?

